

Capítulo 15 — Proyecto Integrador

Sección 01: Introducción al proyecto integrador

A lo largo del Módulo 3 construiste las piezas de un rompecabezas: instrucciones del sistema, ventanas de contexto, memoria persistente, recuperación aumentada por recuperación, herramientas externas, agentes, observabilidad y seguridad. Cada capítulo desarrolló esas piezas por separado, con casos enfocados y ejercicios acotados. Este capítulo las ensambla.

El proyecto integrador no es un repaso ni un resumen. Es una instancia de síntesis activa: diseñarás una solución de IA de extremo a extremo, tomarás decisiones de arquitectura reales con restricciones reales, y documentarás el razonamiento detrás de cada elección. Al final del capítulo tendrás un artefacto completo: una arquitectura de referencia con sus decisiones justificadas, un laboratorio para verificar tu comprensión, y un checklist profesional que podrás aplicar a cualquier proyecto propio.

Qué cubre este capítulo

El capítulo se organiza en cuatro bloques:

Bloque 1 — Marco del proyecto (secciones 01–03). Define el problema de negocio que servirá como hilo conductor, establece los requisitos que la solución debe satisfacer y produce el diagrama de arquitectura completa que integra todos los componentes del módulo.

Bloque 2 — Diseño dimensional (secciones 04–08). Examina cada dimensión técnica por separado: cómo se diseñan el contexto y la memoria, cómo se integran RAG y herramientas, cómo se incorporan los agentes, cómo se instrumenta la observabilidad y la seguridad, y cómo se planifica el despliegue.

Bloque 3 — Revisión crítica y laboratorio (secciones 09–11). Identifica los errores que emergen cuando se integran componentes (distintos de los errores que aparecen en componentes aislados), presenta el caso completo de implementación como referencia, y propone el laboratorio guiado en tres niveles de profundidad.

Bloque 4 — Cierre del módulo (secciones 12–15). Entrega el checklist profesional definitivo, resume los principios fundamentales del Context Engineering, propone criterios de autoevaluación y establece el puente hacia el Módulo 4.

El caso de referencia

Para que el proyecto sea concreto, trabajarás a lo largo de todo el capítulo con un único caso de negocio: el asistente empresarial interno de TechCore S.A., una organización de doscientas personas con departamentos de TI, Legal, Recursos Humanos y Finanzas. Ese caso fue seleccionado porque obliga a usar todos los componentes del módulo: instrucciones del sistema diferenciadas por departamento, memoria de usuario, RAG sobre documentación interna, herramientas de sistemas corporativos, un agente de análisis de incidentes, observabilidad operacional y controles de acceso por rol.

La selección de un caso de complejidad intermedia es deliberada. Un caso trivial —un chatbot de preguntas frecuentes— no requiere agentes ni memoria persistente y no ejercita la arquitectura completa. Un caso de máxima complejidad —un sistema multiagente distribuido globalmente— introduce dificultades de infraestructura que oscurecen las decisiones de Context Engineering. TechCore ocupa el punto medio: suficientemente ambiguo para que las decisiones de diseño no sean únicas ni obvias, suficientemente concreto para que el análisis sea preciso.

Cómo trabajar este capítulo

Cada sección introduce un aspecto del diseño y lo aplica al caso TechCore. El objetivo no es que memorices la arquitectura resultante sino que entiendas el razonamiento que la produce. Cuando leas las justificaciones de cada decisión, pregúntate si en tu propio contexto profesional ese razonamiento cambiaría y por qué. La arquitectura de TechCore es una referencia, no un modelo universal.

El laboratorio integrador (sección 11) está diseñado para que puedas completarlo sin infraestructura de producción. Requiere diagramas, documentación de decisiones y análisis de riesgos, no despliegues en la nube ni acceso a APIs empresariales. Lo que se evalúa es la capacidad de diseñar y justificar, no la de ejecutar.

Con ese marco establecido, la siguiente sección define el problema de negocio con precisión suficiente para que las decisiones de arquitectura tengan base concreta.

Capítulo 15 — Proyecto Integrador

Sección 02: Definición del problema de negocio

El contexto organizacional

TechCore S.A. es una empresa de servicios tecnológicos con doscientos empleados distribuidos en cuatro departamentos operativos: Tecnología de la Información (TI), Legal, Recursos Humanos y Finanzas. La compañía maneja una base documental interna extensa: políticas de seguridad, contratos tipo, manuales de procedimientos, runbooks de operaciones y normativas de cumplimiento. Esa documentación existe en formatos heterogéneos —PDF, documentos de texto, wikis internas, correos archivados— y está dispersa en sistemas distintos.

El problema no es la falta de información. El problema es el costo de acceder a ella. Un analista de TI que necesita el procedimiento de escalación de incidentes críticos puede tardarse veinte minutos en localizar el runbook correcto entre carpetas compartidas. Un empleado nuevo de Recursos Humanos que quiere saber el protocolo de baja por enfermedad consulta a tres personas distintas antes de obtener una respuesta definitiva. Un abogado interno que necesita una cláusula estándar de confidencialidad recorre los contratos archivados a mano.

El proyecto

TechCore decide construir un asistente empresarial interno accesible vía interfaz de chat. El asistente debe responder preguntas operativas de los empleados usando la documentación interna como fuente de verdad, ejecutar acciones en sistemas corporativos cuando el usuario lo solicita, y recordar preferencias y contexto de conversaciones anteriores para no obligar al usuario a repetirse.

Requisitos funcionales

El asistente debe satisfacer los siguientes requisitos para ser considerado una solución completa:

RF-01. Respuestas contextualizadas por departamento. El comportamiento del asistente varía según el departamento del usuario. Un empleado de TI recibe respuestas orientadas a procedimientos técnicos. Un empleado de Legal recibe respuestas con referencias normativas. La personalización del sistema debe ser transparente para el usuario pero configurable por el administrador.

RF-02. Recuperación de documentación interna. El asistente accede a la base documental corporativa y recupera fragmentos relevantes para cada consulta. No genera información que no esté en la documentación: cuando no encuentra respuesta, lo dice explícitamente y sugiere con quién consultar.

RF-03. Ejecución de acciones en sistemas. El asistente puede crear tickets de soporte en el sistema de gestión de incidentes, consultar el directorio de empleados, verificar el estado de solicitudes en curso y agendar recordatorios en el calendario corporativo. Todas las acciones requieren confirmación del usuario antes de ejecutarse.

RF-04. Memoria de usuario. El asistente recuerda, entre sesiones, las preferencias del usuario (idioma, nivel de detalle preferido, rol dentro del departamento) y el contexto de solicitudes en curso. Si un empleado estaba gestionando un incidente la semana anterior, el asistente puede retomar ese hilo cuando el empleado vuelva.

RF-05. Análisis de incidentes de TI. Para el departamento de TI, el asistente puede operar como un agente de análisis: dado un incidente reportado, consulta el historial de incidentes similares, revisa el runbook aplicable, propone pasos de diagnóstico ordenados y escala automáticamente al nivel de soporte correspondiente si el incidente supera los umbrales predefinidos.

RF-06. Control de acceso por departamento y rol. Los documentos de cada departamento son accesibles únicamente para empleados de ese departamento. Los documentos de alta confidencialidad (contratos activos, datos salariales) requieren nivel de autorización adicional. El asistente nunca revela información de un departamento a un usuario que no pertenece a él.

Requisitos no funcionales

RNF-01. Latencia aceptable. Las respuestas simples (preguntas de procedimiento sin herramientas) deben llegar en menos de tres segundos. Las respuestas que implican recuperación de documentos deben llegar en menos de ocho segundos. Las respuestas del agente de análisis de incidentes pueden tomar hasta treinta segundos.

RNF-02. Disponibilidad. El sistema debe estar disponible durante el horario laboral extendido (6:00–22:00 hora local) con un objetivo de disponibilidad del 99 %.

RNF-03. Auditoría. Toda interacción debe quedar registrada con: identificador del usuario, departamento, marca de tiempo, consulta, respuesta y fuentes recuperadas. Los registros deben ser inmutables y conservarse durante doce meses.

RNF-04. Infraestructura accesible. La primera versión del sistema debe poder desplegarse con herramientas de mercado accesibles: un proveedor de LLM mediante API, una base vectorial gestionada, un almacén de clave-valor para la memoria, y servicios de logging estándar. No se asume infraestructura propietaria ni equipos de ML internos.

Decisiones de alcance

El asistente de TechCore v1.0 tiene tres exclusiones explícitas de alcance:

- 1. **No entrena modelos propios.** Usa el LLM mediante API, sin fine-tuning. La personalización se logra exclusivamente vía Context Engineering.
- 2. **No opera en multiagente en la primera versión.** El agente de análisis de incidentes es un agente único con un conjunto fijo de herramientas. La arquitectura multiagente queda fuera del alcance de v1.0.
- 3. **No procesa imágenes ni documentos escaneados no procesados.** La base documental debe estar en texto extraíble. El procesamiento de imágenes y OCR es un requisito de v2.0.

Por qué este caso ejerce todos los componentes del módulo

Componente del módulo	Requisito de TechCore que lo activa
Instrucciones del sistema	RF-01 — comportamiento por departamento
Ventana de contexto	RF-04 — contexto de conversaciones largas
Memoria persistente	RF-04 — memoria entre sesiones
RAG	RF-02 — recuperación de documentación
Herramientas	RF-03 — acciones en sistemas corporativos
Agentes	RF-05 — análisis de incidentes de TI
Observabilidad	RNF-01, RNF-03 — latencia y auditoría
Seguridad	RF-06 — control de acceso por rol

Ningún componente queda sin uso. Esa cobertura completa es el criterio que valida TechCore como caso de referencia para el proyecto integrador.

Con los requisitos definidos, la siguiente sección construye la arquitectura que los satisface y produce el diagrama de referencia del capítulo.

Capítulo 15 — Proyecto Integrador

Sección 03: Diseño de la arquitectura completa

Esta sección es el corazón del capítulo. El diagrama que produce no es un resumen de los capítulos anteriores: es el plano de trabajo que te guiará durante las secciones 04 a 08. Consérvalo como plantilla para tus propios proyectos.

Los siete componentes de la arquitectura

Una solución de IA de producción basada en Context Engineering se organiza en siete componentes interconectados. Cada uno tiene responsabilidades precisas y límites bien definidos.

Componente 1 — Capa de presentación. Es la interfaz a través de la cual el usuario interactúa. Puede ser una interfaz de chat web, una API REST consumida por otras aplicaciones internas o un bot de mensajería corporativa. Su responsabilidad es capturar la entrada del usuario, enviarla al orquestador y renderizar la respuesta. No tiene lógica de negocio.

Componente 2 — Orquestador de contexto. Es el núcleo de la arquitectura. Recibe la entrada del usuario, construye el contexto que se enviará al LLM, coordina la recuperación de memoria y documentos, invoca herramientas cuando el LLM lo solicita, y devuelve la respuesta ensamblada a la capa de presentación. El orquestador es quien toma las decisiones de qué cabe en la ventana de contexto y en qué orden.

Componente 3 — Módulo de instrucciones del sistema. Almacena y sirve las instrucciones del sistema según el perfil del usuario. Cada departamento tiene al menos una instrucción del sistema distinta. El módulo recibe el perfil del usuario (departamento, rol, nivel de autorización) y devuelve la instrucción del sistema correspondiente. Puede almacenar múltiples versiones de la misma instrucción para pruebas A/B.

Componente 4 — Módulo de memoria. Gestiona dos tipos de memoria: memoria de sesión (dentro de una conversación) y memoria persistente (entre conversaciones). Persiste en un almacén de clave-valor. El orquestador consulta este módulo al inicio de cada turno

para recuperar el contexto relevante de conversaciones previas y actualiza el almacén al final del turno con la información nueva que vale la pena conservar.

Componente 5 — Motor de recuperación (RAG). Contiene el pipeline de recuperación aumentada: un índice vectorial con los fragmentos de la documentación interna, el componente de embedding que convierte consultas en vectores, y el motor de búsqueda semántica. Devuelve los fragmentos más relevantes para cada consulta, acompañados de metadatos de origen (documento, sección, fecha de última actualización) que se usan para las citas y para el control de acceso.

Componente 6 — Módulo de herramientas. Expone las acciones que el LLM puede invocar sobre sistemas externos: creación de tickets, consulta del directorio, verificación de solicitudes, agendamiento de recordatorios. Cada herramienta tiene una firma de entrada y salida tipada, validación de permisos, y un mecanismo de confirmación que pausa la ejecución hasta que el usuario apruebe la acción.

Componente 7 — Capa de observabilidad y seguridad. Registra cada evento del sistema: entrada del usuario, contexto enviado al LLM, respuesta recibida, herramientas invocadas, fuentes recuperadas, latencias en cada etapa. También aplica los controles de seguridad: filtrado de salida, validación de referencias de documentos, enforcement de control de acceso, y detección de intentos de prompt injection.

El diagrama de referencia





Las cinco decisiones de arquitectura

Un diagrama sin las decisiones que lo produjeron es incompleto. Cada elección estructural de esta arquitectura tiene una justificación y una alternativa descartada.

Decisión 1: Orquestador propio en lugar de framework de alto nivel.

Se decidió construir el orquestador como componente propio en lugar de usar un framework de agentes de alto nivel (LangChain, LlamaIndex, AutoGPT). La razón es el control de observabilidad: los frameworks de alto nivel ocultan las llamadas al LLM y dificultan el registro granular de cada paso. En un sistema empresarial con requisitos de auditoría (RNF-03), la transparencia de cada operación no es negociable. El costo de esa decisión es mayor tiempo de desarrollo inicial; el beneficio es inspectabilidad completa del flujo.

Decisión 2: Instrucciones del sistema gestionadas como datos, no como código.

Las instrucciones del sistema se almacenan en una base de datos o sistema de archivos versionado, no en el código fuente de la aplicación. Esto permite actualizar el comportamiento del asistente sin re-desplegar el servicio. Un cambio en la política de escalación de TI se refleja actualizando el texto de la instrucción del sistema del perfil TI, no modificando el código. La alternativa —instrucciones embebidas en el código— haría cada ajuste editorial una operación de ingeniería.

Decisión 3: RAG en lugar de fine-tuning para el conocimiento interno.

La documentación interna de TechCore cambia frecuentemente: las políticas se actualizan, los runbooks se modifican, los contratos se renuevan. Fine-tuning de un modelo con documentación interna requeriría re-entrenamiento cada vez que hay un cambio, con los costos y plazos que eso implica. RAG permite actualizar el índice vectorial sin modificar el modelo base y, adicionalmente, provee referencias exactas a los documentos fuente, lo que cumple el requisito de auditoría. Fine-tuning tiene sentido cuando el conocimiento es estable y la velocidad de inferencia es crítica; ninguna de las dos condiciones aplica a TechCore v1.0.

Decisión 4: Un solo agente de análisis de incidentes, no multiagente.

RF-05 podría haberse implementado con una arquitectura multiagente: un agente coordinador que delega en agentes especializados de diagnóstico, búsqueda en runbooks y escalación. Esa arquitectura añade flexibilidad pero también añade puntos de falla, latencia acumulada y complejidad de depuración. Para v1.0, un agente único con herramientas bien definidas es suficiente: puede diagnosticar, buscar en runbooks y escalar. La transición a multiagente se justificará cuando los ciclos de razonamiento del agente único sean insuficientes para los incidentes más complejos.

Decisión 5: Confirmación de usuario antes de ejecutar herramientas.

Toda acción sobre sistemas externos requiere confirmación explícita del usuario. Esta decisión sacrifica velocidad de interacción a favor de control: el usuario siempre sabe qué va a hacer el sistema antes de que lo haga. Un sistema que crea tickets o agenda reuniones sin confirmación puede generar efectos no deseados difíciles de revertir. La confirmación es el mecanismo de supervisión humana en el nivel de interacción.

La arquitectura como contrato

El diagrama y las cinco decisiones funcionan como un contrato entre el equipo de ingeniería, los usuarios del sistema y los administradores: definen qué hace el sistema, qué no hace, y por qué. Un contrato arquitectónico documentado es la diferencia entre un sistema que el equipo puede mantener y evolucionar de manera razonada, y un sistema que crece de manera improvisada hasta volverse inmanejable.

Las secciones que siguen desarrollan cada componente con el nivel de detalle necesario para pasar del diagrama a la implementación.

Capítulo 15 — Proyecto Integrador

Sección 04: Diseño del contexto y la memoria

Los capítulos 04 al 07 del módulo establecieron los fundamentos: cómo funciona la ventana de contexto, qué tipos de memoria existen, cómo se gestiona la información entre turnos. Esta sección aplica esos fundamentos al diseño concreto de TechCore.

La estructura del contexto por turno

Cada vez que el usuario envía un mensaje, el orquestador construye el contexto que se enviará al LLM. Ese contexto tiene una estructura fija con cuatro zonas:

ZONA 1 — INSTRUCCIÓN DEL SISTEMA
(fija por sesión, varía por departamento y rol)
Aprox. 800-1200 tokens
ZONA 2 — CONTEXTO DE MEMORIA PERSISTENTE
(recuperado al inicio de la sesión)
Aprox. 400-600 tokens
ZONA 3 — FRAGMENTOS RAG
(recuperados por consulta, turno a turno)
Aprox. 1500-2500 tokens
ZONA 4 — HISTORIAL DE CONVERSACIÓN
(ventana deslizante de los últimos N turnos)
Aprox. 1000-2000 tokens

El presupuesto total de tokens por turno para TechCore v1.0 es de 6.000 tokens de entrada, reservando el espacio restante de la ventana del modelo (típicamente 128.000 tokens en

modelos actuales) para futuras expansiones. Esta limitación autoimpuesta evita que el costo de la API escale sin control en las primeras semanas de operación.

Diseño de las instrucciones del sistema por departamento

Cada perfil departamental tiene una instrucción del sistema que responde a cuatro preguntas: quién es el asistente en este contexto, qué puede hacer, qué no puede hacer, y cómo debe responder.

Instrucción del sistema — Perfil TI (extracto):

```
Eres el asistente interno de TI de TechCore. Tu función principal es
ayudar a los ingenieros y analistas de soporte a resolver incidentes,
localizar procedimientos en los runbooks de operaciones, y gestionar
solicitudes en el sistema de tickets.
```

Puedes:

- Responder preguntas técnicas de procedimiento con base en la documentación de TI disponible.
- Crear y consultar tickets en el sistema de gestión de incidentes.
- Proponer pasos de diagnóstico para incidentes de infraestructura.
- Escalar automáticamente incidentes P1 al equipo de guardia cuando el usuario lo confirme.

No puedes:

- Acceder a información de otros departamentos (Legal, RRHH, Finanzas).
- Ejecutar comandos directamente en sistemas de producción.
- Proporcionar credenciales de acceso bajo ninguna circunstancia.

Nivel de detalle: técnico. Usa terminología de operaciones de TI.

Cuando cites un runbook, incluye el nombre del documento y la sección.

Instrucción del sistema — Perfil Legal (extracto):

Eres el asistente interno de Legal de TechCore. Tu función principal es ayudar a los abogados internos y al equipo de cumplimiento a localizar cláusulas en contratos tipo, interpretar políticas internas con base en su texto, y gestionar solicitudes de revisión documental.

Puedes:

- Recuperar y citar cláusulas de los contratos tipo disponibles.
- Responder preguntas sobre políticas internas citando el documento fuente.
- Crear solicitudes de revisión de documentos.

No puedes:

- Proporcionar asesoría legal vinculante. Siempre indica que las respuestas son orientativas y recomienda consultar al abogado responsable para decisiones con efecto legal.
- Acceder a contratos activos en negociación sin nivel de autorización elevado.

Nivel de detalle: preciso. Cita textualmente cuando sea posible.

Indica siempre la versión y fecha del documento recuperado.

Los perfiles de RRHH y Finanzas siguen la misma estructura con restricciones específicas de sus dominios (datos salariales, información médica, datos de nómina).

Gestión de la ventana deslizante

La ventana de conversación (Zona 4) usa una estrategia de ventana deslizante con resumen incremental. Las últimas cinco interacciones se conservan en texto completo. Las interacciones anteriores se comprimen en un resumen que el orquestador genera y almacena al momento de la compresión.

Esta elección tiene una consecuencia importante: el resumen es una interpretación, no una copia fiel. Si el usuario necesita recuperar un texto exacto de una conversación anterior, el sistema no puede garantizarlo. El diseño acepta ese límite a cambio de poder sostener conversaciones largas sin que el costo de tokens se vuelva prohibitivo.

El algoritmo de ventana deslizante para TechCore:


```
def construir_historial(turnos: list[Turno], presupuesto_tokens: int) -> str:
    """
    Devuelve el historial de conversación que cabe en el presupuesto.
    Conserva los últimos 5 turnos completos.
    Comprime el resto en un resumen.
    """
    TURNOS_COMPLETOS = 5

    if len(turnos) <= TURNOS_COMPLETOS:
        return formatear_turnos(turnos)

    turnos_recientes = turnos[-TURNOS_COMPLETOS:]
    turnos_anteriores = turnos[:-TURNOS_COMPLETOS]

    resumen = recuperar_resumen_acumulado(turnos_anteriores)
    if resumen is None:
        resumen = generar_resumen(turnos_anteriores)
        almacenar_resumen(resumen)

    return f"[Resumen de conversación previa]\n{resumen}\n\n" + \
        formatear_turnos(turnos_recientes)
```

Diseño del módulo de memoria persistente

La memoria persistente de TechCore almacena información que tiene valor más allá de una sesión. No todo lo que ocurre en una conversación merece ser memorizado: el criterio de selección es si la información cambiaría el comportamiento del asistente en la siguiente sesión.

Qué se memoriza:

Tipo de información	Ejemplo	TTL
Preferencias del usuario	"Prefiero respuestas en viñetas"	Sin expiración
Rol dentro del departamento	"Soy el responsable de turno nocturno de TI"	90 días
Solicitudes en curso	"Tengo abierto el ticket #4521"	Hasta resolución
Incidentes recientes	"El servidor web-03 estuvo caído el martes"	30 días

Qué no se memoriza:

- El contenido literal de las respuestas del asistente.
- Datos personales sensibles (salud, situación financiera personal).
- Información que el usuario no proveyó explícitamente (inferencias especulativas).

Estructura de entrada en el KV store:

```
{
  "user_id": "emp-0472",
  "department": "ti",
  "memories": [
    {
      "key": "preferencia_formato",
      "value": "viñetas cortas",
      "source": "explícita",
      "timestamp": "2026-06-15T09:23:00Z",
      "ttl_days": null
    },
    {
      "key": "tickets_activos",
      "value": ["#4521", "#4589"],
      "source": "inferida_de_conversación",
      "timestamp": "2026-07-20T14:15:00Z",
      "ttl_days": 30
    }
  ]
}
```

El contexto de memoria en el prompt

Al inicio de cada sesión, el orquestador recupera la memoria persistente del usuario y la inyecta en la Zona 2 del contexto con un encabezado explícito:

```
[CONTEXTO DEL USUARIO - emp-0472]

Departamento: TI | Rol: Responsable de turno nocturno

Preferencias: respuestas en viñetas cortas, terminología técnica

Solicitudes activas: Ticket #4521 (disco lleno web-03, abierto hace 3 días)

Última sesión: 2026-07-22 — consultó runbook de escalación de incidentes P1
```

Este encabezado informa al LLM del estado actual del usuario sin que el usuario tenga que repetirlo. Si el usuario pregunta "¿cómo va lo del ticket?", el LLM ya sabe a qué ticket se refiere.

Límites y compromisos de diseño

El diseño de memoria de TechCore asume que la memoria es un ayudante, no un archivo. Si el usuario quiere que el asistente olvide algo, puede solicitarlo explícitamente y el sistema debe honrarlo borrando la entrada del KV store. Este comportamiento no es solo una cortesía: en entornos regulados, el derecho al olvido puede ser un requisito legal.

La siguiente sección construye sobre esta base de contexto y memoria para integrar la recuperación de documentos y las herramientas externas.

Capítulo 15 — Proyecto Integrador

Sección 05: Integración de RAG, herramientas y MCP

La memoria del sistema, desarrollada en la sección anterior, provee el contexto del usuario. RAG provee el conocimiento corporativo. Las herramientas proveen la capacidad de actuar sobre sistemas externos. Esta sección diseña la integración de esos tres componentes en el orquestador de TechCore.

El pipeline de recuperación de documentos

El motor RAG de TechCore tiene cinco etapas secuenciales, cada una con un criterio de calidad específico.

Etapas 1 — Ingestión y chunking. Los documentos de la base documental corporativa se procesan en fragmentos (chunks) de aproximadamente 400 tokens, con solapamiento de 80 tokens entre fragmentos consecutivos. El solapamiento evita que un concepto que aparece en la frontera entre dos fragmentos quede sin contexto en ninguno de los dos.

Los metadatos de cada fragmento incluyen: nombre del documento, sección, fecha de última modificación, departamento propietario y nivel de clasificación (público interno / confidencial / restringido). Estos metadatos son la base del control de acceso en el nivel de recuperación.

```
class FragmentoDocumental:

    texto: str          # contenido del fragmento
    embedding: list      # vector de 1536 dimensiones
    doc_nombre: str      # "Runbook-Incidentes-TI-v4.2.pdf"
    seccion: str         # "3.2 Escalación a soporte nivel 2"
    fecha_mod: date      # 2026-05-10
    departamento: str    # "ti"
    clasificacion: str    # "público_interno"
```

Etapla 2 — Generación de embeddings. Cada fragmento se convierte en un vector mediante el modelo de embeddings. Para TechCore, se usa el mismo proveedor de API que el LLM, asegurando compatibilidad y un único punto de gestión de credenciales. Los embeddings se generan una vez en la ingestión y se re-generan únicamente cuando el documento fuente cambia.

Etapla 3 — Recuperación semántica. Dado el mensaje del usuario, el orquestador genera el embedding de la consulta y ejecuta una búsqueda de similitud coseno en el índice vectorial. Se recuperan los k fragmentos más similares, donde k se fija en 5 para TechCore v1.0. Este valor de k es conservador: garantiza que los fragmentos RAG no excedan el presupuesto de la Zona 3 del contexto (aproximadamente 2.500 tokens).

Etapla 4 — Filtrado por control de acceso. Antes de incluir los fragmentos recuperados en el contexto, el orquestador verifica que el usuario tiene permiso para acceder al departamento propietario de cada fragmento. Un fragmento del runbook de TI nunca llega al contexto de un usuario de Legal, aunque sea semánticamente relevante para su consulta. Los fragmentos filtrados se descartan sin notificación al usuario (el usuario no debe saber qué documentos confidenciales existen).

Etapla 5 — Construcción de la Zona 3. Los fragmentos que superan el filtro de acceso se incluyen en el contexto con un formato explícito que le indica al LLM su naturaleza y origen:

```
[DOCUMENTACIÓN INTERNA RECUPERADA]
```

```
Fuente 1: Runbook-Incidentes-TI-v4.2.pdf — Sección 3.2 (actualizado 2026-05-10)
```

```
"El escalamiento a soporte de nivel 2 se activa cuando el tiempo de resolución supera los 45 minutos o cuando el impacto afecta a más de 10 usuarios..."
```

```
Fuente 2: Política-Gestión-Incidentes-v2.1.pdf — Sección 5 (actualizado 2026-03-22)
```

```
"Los incidentes P1 requieren notificación al Gerente de TI en un plazo máximo de 15 minutos desde la detección..."
```

```
[FIN DOCUMENTACIÓN RECUPERADA]
```

El formato explícito tiene dos funciones: le indica al LLM cuándo citar sus fuentes, y le permite al sistema de observabilidad extraer qué documentos contribuyeron a cada respuesta.

Herramientas disponibles

TechCore v1.0 expone cuatro herramientas al LLM. Cada herramienta tiene una especificación completa de entrada, salida y condiciones de uso.

Herramienta 1: `crear_ticket`

```
{
  "nombre": "crear_ticket",
  "descripcion": "Crea un nuevo ticket en el sistema de gestión de incidentes.
                  Solo disponible para usuarios de TI. Requiere confirmación
                  del usuario antes de ejecutarse.",
  "parametros": {
    "tipo": "incidente | solicitud | cambio",
    "titulo": "string (max 100 caracteres)",
    "descripcion": "string (max 1000 caracteres)",
    "prioridad": "P1 | P2 | P3 | P4",
    "asignado_a": "string (email) | null"
  },
  "respuesta": {
    "ticket_id": "string",
    "url": "string",
    "estado": "creado"
  },
  "permisos_requeridos": ["ti"],
  "requiere_confirmacion": true
}
```

Herramienta 2: `consultar_directorio`

```
{
  "nombre": "consultar_directorio",
  "descripcion": "Busca empleados en el directorio corporativo por nombre
                  o departamento. Disponible para todos los usuarios.",
  "parametros": {
    "nombre": "string | null",
    "departamento": "ti | legal | rrhh | finanzas | null"
  },
  "respuesta": {
    "empleados": [
      {
        "nombre": "string",
        "cargo": "string",
        "email": "string",
        "departamento": "string"
      }
    ]
  },
  "permisos_requeridos": [],
  "requiere_confirmacion": false
}
```

Herramienta 3: `verificar_solicitud`

```
{
  "nombre": "verificar_solicitud",
  "descripcion": "Consulta el estado actual de una solicitud o ticket existente.",
  "parametros": {
    "id_solicitud": "string"
  },
  "respuesta": {
    "id": "string",
    "estado": "abierto | en_progreso | resuelto | cerrado",
    "titulo": "string",
    "creado": "datetime",
    "ultima_actualización": "datetime",
    "asignado_a": "string | null"
  },
  "permisos_requeridos": [],
  "requiere_confirmacion": false
}
```

Herramienta 4: `agendar_recordatorio`

```
{
  "nombre": "agendar_recordatorio",
  "descripcion": "Crea un recordatorio en el calendario del usuario.",
  "parametros": {
    "fecha_hora": "datetime (ISO 8601)",
    "descripcion": "string (max 200 caracteres)"
  },
  "respuesta": {
    "id_recordatorio": "string",
    "confirmado": true
  },
  "permisos_requeridos": [],
  "requiere_confirmacion": true
}
```


El ciclo de invocación de herramientas

El LLM no ejecuta herramientas directamente: solicita al orquestador que las ejecute. El flujo es:

Usuario: "Crea un ticket P1 por el servidor web-03 que está caído"

|

▼

Orquestador → LLM (con contexto completo)

|

▼

LLM responde con llamada a herramienta:

```
{  
  "tool": "crear_ticket",  
  "parametros": {  
    "tipo": "incidente",  
    "titulo": "Servidor web-03 caído",  
    "descripcion": "El servidor web-03 no responde. Impacto: producción.",  
    "prioridad": "P1"  
  }  
}
```

|

▼

Orquestador valida permisos → usuario pertenece a TI ✓

|

▼

Orquestador presenta al usuario:

"Voy a crear el siguiente ticket:

- Tipo: Incidente P1
- Título: Servidor web-03 caído
- Descripción: El servidor web-03 no responde. Impacto: producción.

¿Confirmas? [Sí / No / Modificar]"

|

▼ (usuario confirma)

Orquestador ejecuta herramienta → ticket_id: #4521

|

▼

Resultado enviado al LLM como mensaje de herramienta

|

▼

```
LLM genera respuesta final:  
"Ticket #4521 creado correctamente. Puedes seguirlo aquí: [url].  
Al ser P1, el equipo de guardia recibirá notificación automática."
```

MCP como capa de integración

El protocolo MCP (Model Context Protocol) es el mecanismo estándar para que el orquestador y el LLM intercambien definiciones de herramientas y resultados de ejecución en un formato estructurado. Para TechCore, MCP resuelve tres problemas:

Primero, estandariza la descripción de herramientas: en lugar de que cada proveedor de LLM tenga su propio formato de function calling, MCP provee un esquema común que puede traducirse a cualquier proveedor. Esto hace que el orquestador no esté acoplado a un único proveedor de API.

Segundo, gestiona el ciclo de vida de las llamadas: MCP define claramente cuándo el LLM ha terminado de razonar y cuándo está esperando el resultado de una herramienta, lo que simplifica el bucle de control del orquestador.

Tercero, permite exponer servidores MCP externos: si TechCore decide integrar el sistema de tickets de un proveedor que ya tiene un servidor MCP publicado, puede conectarlo sin escribir código de integración a medida.

La implementación de MCP en TechCore v1.0 usa el cliente MCP estándar en el orquestador, con servidores MCP locales para cada herramienta corporativa. Los servidores locales hacen las llamadas reales a las APIs internas de TechCore y devuelven resultados en formato MCP al orquestador.

Degradación controlada

Cuando una herramienta falla —el sistema de tickets no responde, el directorio está en mantenimiento— el sistema no debe fallar silenciosamente. El diseño de TechCore incluye respuestas de degradación controlada para cada herramienta:

Herramienta	Fallo	Respuesta al usuario
<code>crear_ticket</code>	API no disponible	"El sistema de tickets no responde. Intenta en [URL] directamente o contacta a soporte."
<code>consultar_directorio</code>	Timeout	"El directorio no está disponible ahora. El área de RRHH puede ayudarte."
<code>verificar_solicitud</code>	Registro no encontrado	"No encontré la solicitud [id]. Verifica el identificador o consulta directamente."
<code>agendar_recordatorio</code>	Error de calendario	"No pude agendar el recordatorio. Puedes hacerlo manualmente desde [URL]."

La degradación controlada asegura que un fallo de herramienta no deje al usuario sin respuesta: el sistema siempre le indica qué alternativa tiene.

Con el diseño de RAG y herramientas establecido, la siguiente sección incorpora el componente más complejo de la arquitectura: el agente de análisis de incidentes de TI.

Capítulo 15 — Proyecto Integrador

Sección 06: Incorporación de agentes y planificación

Las secciones anteriores construyeron un asistente capaz de responder preguntas y ejecutar acciones simples con confirmación del usuario. Ese asistente es un sistema reactivo: responde a lo que el usuario pregunta. El agente de análisis de incidentes de TI introduce una capacidad cualitativamente distinta: el sistema puede razonar en múltiples pasos, consultar herramientas en secuencia y tomar decisiones intermedias sin input del usuario en cada paso.

Cuándo un agente es la respuesta correcta

Antes de diseñar el agente, vale la pena enunciar explícitamente por qué RF-05 requiere un agente y no un flujo simple de herramienta.

Un incidente de TI no es una consulta con una respuesta directa. Es un proceso de investigación: se observa un síntoma, se formulan hipótesis, se recopila información para verificarlas, se descarta lo que no aplica, y se converge en un diagnóstico. Ese proceso tiene ramificaciones: la secuencia de pasos depende de lo que se encuentra en cada paso. Un flujo fijo de herramientas no puede manejar esa condicionalidad. Un agente, en cambio, puede razonar sobre lo que encontró y decidir qué herramienta invocar a continuación.

La heurística práctica: si la secuencia de herramientas que se necesita para resolver la tarea no puede definirse antes de ejecutarla, se necesita un agente.

Arquitectura del agente de incidentes

El agente de incidentes opera bajo el ciclo ReAct (Reason-Act-Observe): razona sobre el estado actual, decide una acción, la ejecuta, observa el resultado, y vuelve a razonar con la nueva información. El orquestador implementa ese ciclo con un límite de iteraciones para evitar bucles infinitos.

CICLO REACT DEL AGENTE DE INCIDENTES

Entrada: descripción del incidente + contexto del usuario

Iteración 1:

Razonamiento: "El usuario reporta que web-03 no responde.

Necesito saber más: cuánto tiempo lleva caído,
si hay otros servicios afectados."

Acción: verificar_solicitud(#4521) – recuperar historial del ticket

Observación: "Ticket abierto hace 47 minutos. Severidad inicial: P2.
Sin actualizaciones desde apertura."

Iteración 2:

Razonamiento: "47 minutos sin resolución. Según política, P2 sin
resolución en 45 min debe escalar a P1. Voy a buscar
el runbook de escalación para verificar."

Acción: RAG("escalación incidentes P2 a P1 tiempo de resolución")

Observación: Fragmento del Runbook-Incidentes-TI-v4.2, sección 4.1:
"Si un incidente P2 no tiene resolución confirmada en 45
minutos, debe reclasificarse a P1 y notificar al Gerente..."

Iteración 3:

Razonamiento: "Confirmado por el runbook. Debo proponer la reclasificación.
Antes de hacerlo, verifico si hay incidentes similares
recientes que puedan indicar la causa raíz."

Acción: RAG("servidor web-03 incidentes previos disco lleno")

Observación: Fragmento del historial de incidentes: "Ticket #4389 (2026-06-28):
disco /var/log lleno en web-03, resuelto con limpieza de logs."

Iteración 4:

Razonamiento: "Hay un antecedente de disco lleno en el mismo servidor.
Es la hipótesis más probable. Voy a formular el diagnóstico
y las acciones recomendadas."

Acción: NINGUNA (síntesis final)

```
Salida al usuario:

Diagnóstico probable: disco lleno en web-03 (antecedente el 28/06).

Acciones recomendadas:

1. Verificar uso de disco en web-03: df -h

2. Si /var/log está al 90%+, ejecutar: journalctl --vacuum-size=500M

3. Reclasificar ticket #4521 de P2 a P1 (47 min sin resolución).

¿Procedo con la reclasificación del ticket? [Sí / No]
```

El agente llegó a un diagnóstico específico usando cuatro pasos de razonamiento, dos consultas al motor RAG y una lectura del sistema de tickets, sin que el usuario tuviera que guiarlo paso a paso. Eso es lo que distingue al agente del asistente reactivo.

Diseño de las herramientas del agente

El agente de incidentes tiene acceso a las mismas herramientas del asistente general más dos adicionales específicas para el análisis de incidentes:

Herramienta 5: `buscar_incidentes_similares`

```
{
  "nombre": "buscar_incidentes_similares",
  "descripcion": "Busca en el historial de incidentes cerrados usando
                  texto libre. Devuelve los 3 incidentes más similares.",
  "parametros": {
    "descripcion": "string",
    "servidor": "string | null",
    "dias_atras": "int (default: 90)"
  },
  "respuesta": {
    "incidentes": [
      {
        "id": "string",
        "titulo": "string",
        "causa_raiz": "string",
        "resolucion": "string",
        "fecha": "date"
      }
    ]
  },
  "permisos_requeridos": ["ti"],
  "requiere_confirmacion": false
}
```

Herramienta 6: `escalar_incidente`


```
{
  "nombre": "escalar_incidente",
  "descripcion": "Reclasifica un incidente existente a mayor prioridad
                  y notifica al equipo de guardia.",
  "parametros": {
    "ticket_id": "string",
    "nueva_prioridad": "P1",
    "justificacion": "string"
  },
  "respuesta": {
    "ticket_id": "string",
    "prioridad_anterior": "string",
    "prioridad_nueva": "P1",
    "notificados": ["string"]
  },
  "permisos_requeridos": ["ti"],
  "requiere_confirmacion": true
}
```

Límites del agente y supervisión humana

El agente de incidentes tiene tres restricciones de diseño que limitan su autonomía:

Restricción 1: máximo de iteraciones. El ciclo ReAct tiene un límite de 8 iteraciones por ejecución. Si el agente no converge en un diagnóstico dentro de ese límite, presenta un resumen de lo que encontró y escala a un humano. Esto evita que el agente consuma tokens indefinidamente en casos de ambigüedad extrema.

Restricción 2: confirmación antes de escalación. El agente puede razonar, diagnosticar y proponer, pero la acción de escalar un incidente (que genera notificaciones al equipo de guardia) siempre requiere confirmación del usuario. El agente nunca escala sin aprobación humana.

Restricción 3: sin ejecución de comandos en producción. El agente puede proponer comandos de diagnóstico (como `df -h` o `journalctl`), pero no puede ejecutarlos directamente en servidores. La ejecución remota de comandos es una capacidad que está fuera del alcance de v1.0 por sus implicaciones de seguridad.

Por qué un solo agente y no multiagente

La decisión de usar un solo agente fue establecida en la sección 03 y vale la pena desarrollarla aquí con el detalle que ese contexto no permitía.

Una arquitectura multiagente para el análisis de incidentes podría verse así: un agente coordinador que recibe el incidente y delega en agentes especializados (agente de diagnóstico, agente de búsqueda en runbooks, agente de escalación). Esa arquitectura tiene dos ventajas: mayor paralelismo (los agentes especializados pueden trabajar en simultáneo) y mayor modularidad (cada agente se puede mejorar independientemente).

Sin embargo, introduce dos costos que v1.0 no puede absorber: primero, la depuración se vuelve significativamente más difícil porque el estado del proceso está distribuido entre múltiples agentes; segundo, el costo de tokens se multiplica porque cada agente recibe su propio contexto completo.

Para TechCore v1.0, el agente único es la elección correcta porque los incidentes que el sistema debe manejar tienen una complejidad media: cuatro iteraciones de razonamiento son suficientes para la mayoría. Cuando el sistema madure y se identifiquen casos que requieren razonamiento más profundo, la migración a multiagente es el camino natural.

La instrucción del sistema del agente

El agente de incidentes usa una instrucción del sistema diferente a la del asistente general de TI. Es más específica en cuanto al proceso que debe seguir:

Eres el agente de análisis de incidentes de TechCore TI. Cuando se te presenta un incidente, sigues este proceso:

1. **RECOPILACIÓN:** Obtén información básica del incidente (estado actual, tiempo transcurrido, impacto).
2. **DIAGNÓSTICO:** Busca incidentes similares anteriores y runbooks aplicables.
3. **HIPÓTESIS:** Formula la causa raíz más probable con base en la evidencia.
4. **PROPUESTA:** Presenta diagnóstico, acciones recomendadas y, si aplica, propuesta de escalación con justificación.

Reglas:

- Nunca ejecutes acciones en producción. Solo propone.
- Para escalar, siempre pide confirmación explícita.
- Si no tienes suficiente información después de 4 pasos, di qué información adicional necesitas y quién puede proveerla.
- Cita siempre el runbook o el incidente previo que respalde tu análisis.

Esta instrucción transforma al LLM de un generador de texto en un agente con un proceso de trabajo explícito. El LLM sabe que debe seguir esos pasos en orden, y el operador que mantiene el sistema sabe exactamente qué comportamiento esperar.

Con el diseño de agentes completo, la siguiente sección incorpora las dos dimensiones operacionales que hacen que un sistema como este sea sostenible en producción: observabilidad y seguridad.

Capítulo 15 — Proyecto Integrador

Sección 07: Observabilidad y seguridad de la solución

Observabilidad y seguridad no son capas que se agregan al final del diseño. Son dimensiones que se planifican desde el primer diagrama de arquitectura. En el caso de TechCore, ambas son requisitos explícitos del pliego: auditoría completa de interacciones (RNF-03) y control de acceso por departamento y rol (RF-06). Esta sección traduce esos requisitos en componentes concretos del sistema.

La capa de observabilidad

La observabilidad de un sistema de IA tiene tres niveles de granularidad, y TechCore los implementa todos.

Nivel 1 — Trazas de interacción. Cada conversación genera una traza que registra la secuencia completa de eventos: entrada del usuario, instrucción del sistema utilizada, fragmentos RAG recuperados, herramientas invocadas y resultados, respuesta del LLM, respuesta final al usuario. Una traza es el registro forense de lo que pasó en una interacción.

Nivel 2 — Métricas de operación. Resúmenes cuantitativos del comportamiento del sistema en el tiempo: latencia por etapa, tasa de recuperación de documentos relevantes, tasa de uso de herramientas, tasa de errores, costo de tokens por usuario y por departamento.

Nivel 3 — Alertas operacionales. Umbrales configurados sobre las métricas que disparan notificaciones cuando el sistema se comporta fuera de los parámetros normales.

Estructura de la traza de interacción

Cada evento en el sistema emite un registro con la siguiente estructura:

```
{
  "trace_id": "trc-20260725-0842-emp0472",
  "session_id": "ses-20260725-0831",
  "user_id": "emp-0472",
  "department": "ti",
  "timestamp": "2026-07-25T08:42:17.334Z",
  "event_type": "interaction_complete",
  "latency_ms": {
    "total": 4821,
    "memory_retrieval": 43,
    "rag_query": 312,
    "llm_inference": 4290,
    "tool_execution": 0
  },
  "context_tokens": {
    "system_prompt": 987,
    "memory": 412,
    "rag_fragments": 1843,
    "conversation_history": 1102,
    "total_input": 4344,
    "output": 287
  },
  "rag": {
    "query_embedding_generated": true,
    "fragments_retrieved": 5,
    "fragments_after_access_filter": 5,
    "documents_cited": [
      "Runbook-Incidentes-TI-v4.2.pdf",
      "Política-Gestión-Incidentes-v2.1.pdf"
    ]
  },
  "tools_invoked": [],
  "response_summary": "Diagnóstico: disco lleno probable. Acciones recomendadas."
}
```

Este registro es inmutable: se escribe una vez y no puede modificarse. El sistema de almacenamiento de logs garantiza integridad mediante hashing del contenido al momento de escritura. El período de retención es de doce meses, conforme a RNF-03.

Las métricas operacionales de TechCore

TechCore monitorea nueve métricas principales. Están agrupadas por dimensión de calidad.

Dimensión de latencia:

Métrica	Objetivo	Alerta si supera
Latencia total P50	< 2.5 s	3.0 s
Latencia total P95	< 6.0 s	8.0 s
Latencia de inferencia LLM	< 4.0 s	6.0 s
Latencia de recuperación RAG	< 500 ms	1.0 s

Dimensión de calidad de recuperación:

Métrica	Objetivo	Alerta si baja de
Fragmentos recuperados con cita	> 70 %	50 %
Respuestas con documentos fuente	> 80 %	60 %

Dimensión de costo:

Métrica	Objetivo	Alerta si supera
Tokens promedio por interacción	< 5.000	6.500
Costo diario por departamento	Según budget	+20 % del budget

Dimensión de error:

Métrica	Objetivo	Alerta si supera
Tasa de errores de herramienta	< 2 %	5 %

El panel de monitoreo

El equipo de operaciones de TechCore accede a un panel que agrega las métricas anteriores en tiempo real. El panel tiene cuatro vistas:

Vista 1 — Salud general del sistema: Estado de cada componente (orquestador, módulo RAG, módulo de herramientas, módulo de memoria), latencia actual P50 y P95, y contador de errores en la última hora.

Vista 2 — Uso por departamento: Volumen de interacciones por departamento, costo de tokens acumulado en el período, herramientas más utilizadas y documentos más recuperados.

Vista 3 — Calidad de respuestas: Tasa de citas de documentos, tasa de uso de herramientas, distribución de longitud de respuestas.

Vista 4 — Alertas activas: Lista de alertas disparadas con severidad, descripción y tiempo transcurrido desde la alerta.

La capa de seguridad

La seguridad del sistema tiene cuatro mecanismos, cada uno diseñado para un tipo de amenaza distinto.

Mecanismo 1 — Control de acceso en el nivel de recuperación. Ya descrito en la sección 05: los fragmentos RAG se filtran por departamento del usuario antes de incluirse en el contexto. El LLM nunca recibe en su contexto documentos a los que el usuario no tiene acceso.

La implementación usa una capa de metadatos en el índice vectorial. Cada fragmento tiene un campo `departamento` y `clasificacion`. La búsqueda vectorial se ejecuta sin filtros para maximizar la relevancia semántica, pero los resultados se filtran en el orquestador antes de construir el contexto:

```
def recuperar_fragmentos_con_control_acceso(
    query: str,
    perfil_usuario: PerfilUsuario,
    k: int = 5
) -> list[FragmentoDocumental]:

    # Recuperar candidatos por similitud semántica
    candidatos = indice_vectorial.buscar(query, k=k*3) # búsqueda amplia

    # Filtrar por permisos del usuario
    autorizados = [
        f for f in candidatos
        if f.departamento in perfil_usuario.departamentos_autorizados
        and nivel_clasificacion_permitido(f.clasificacion, perfil_usuario.nivel)
    ]

    # Devolver los k más similares que pasaron el filtro
    return autorizados[:k]
```

Mecanismo 2 — Filtrado de salida. La respuesta del LLM pasa por un filtro antes de llegar al usuario. El filtro verifica:

- Que no contiene datos personales sensibles (números de identificación, salarios, información médica) que el usuario no debería ver.
- Que las citas de documentos corresponden a fragmentos que efectivamente fueron recuperados en ese turno (evita que el LLM invente referencias).
- Que no contiene instrucciones que parecen ser el resultado de una inyección de prompt exitosa.

Mecanismo 3 — Detección de prompt injection. El módulo de seguridad aplica un conjunto de heurísticas sobre la entrada del usuario para detectar intentos de manipulación del sistema:


```

PATRONES_INJECTION = [
    r"ignora las instrucciones anteriores",
    r"olvida todo lo que te dijeron",
    r"actúa como si fueras",
    r"DAN mode",
    r"jailbreak",
    r"modo desarrollador",
    r"eres ahora un",
    r"tu instrucción real es",
]

def detectar_injection(entrada: str) -> DetectionResult:
    for patron in PATRONES_INJECTION:
        if re.search(patron, entrada, re.IGNORECASE):
            return DetectionResult(
                detectado=True,
                patron=patron,
                accion="bloquear_y_registrar"
            )
    return DetectionResult(detectado=False)

```

Cuando se detecta un intento de inyección, el sistema bloquea la interacción, registra el evento con la entrada completa del usuario, y devuelve un mensaje genérico: "No puedo procesar esa solicitud." El registro del evento incluye la entrada completa para análisis posterior.

Mecanismo 4 — Auditoría de acceso a documentos confidenciales. El acceso a documentos de clasificación `restringido` (contratos activos, datos salariales) genera un evento de auditoría adicional que se envía a un canal separado con alertas en tiempo real al responsable de cumplimiento. Este nivel de alerta no existe para documentos de clasificación `público_interno`.

Seguridad desde el diseño, no desde el parche

El principio que guía el diseño de seguridad de TechCore es que ningún mecanismo de seguridad compensa un diseño arquitectónico inseguro. Si los documentos confidenciales fueran accesibles sin filtro y dependiéramos únicamente del filtrado de salida para

protegerlos, el sistema sería fundamentalmente inseguro aunque el filtro funcionara bien el 99 % del tiempo. El 1 % restante sería un incidente de exposición de datos.

El orden correcto es: primero, no permitir que información no autorizada llegue al contexto del LLM; segundo, verificar que la salida del LLM no contenga información no autorizada como segunda línea de defensa. Los dos mecanismos juntos son robustos. Solo el segundo no lo es.

Con la observabilidad y la seguridad diseñadas, la siguiente sección cierra el bloque de diseño dimensional con la estrategia de despliegue y operación del sistema.

Capítulo 15 — Proyecto Integrador

Sección 08: Estrategia de despliegue y operación

Un sistema bien diseñado que no puede desplegarse de manera confiable no tiene valor en producción. Esta sección define la estrategia de despliegue de TechCore: cómo se lleva el sistema de desarrollo a producción, cómo se mantiene operativo, y cómo se gestiona su evolución después del lanzamiento inicial.

El principio del despliegue incremental

TechCore no se despliega en un único momento para todos sus usuarios. La estrategia es incremental: se valida en contextos controlados antes de exponer el sistema a toda la organización. Esto reduce el riesgo de un fallo masivo y permite recoger retroalimentación real antes de que el sistema opere a escala completa.

La estrategia de despliegue tiene cuatro fases:

Fase 0 — Entorno de desarrollo. El sistema opera en un entorno aislado con datos ficticios. Los desarrolladores interactúan con él para verificar el comportamiento funcional de cada componente. No hay usuarios reales. La base documental RAG contiene documentos de prueba, no la documentación corporativa real.

Fase 1 — Piloto con equipo de TI (semanas 1–4). Se despliega el sistema con datos reales solo para el equipo de TI, que es el departamento con el caso de uso más complejo (el agente de incidentes). El objetivo de esta fase es identificar fallas en el pipeline de RAG con documentación real, validar que el agente de incidentes se comporta correctamente, y calibrar los umbrales de latencia con carga real. Al final de la fase 1, se tiene un sistema funcional validado con un departamento.

Fase 2 — Expansión a todos los departamentos (semanas 5–8). Se onboardean los perfiles de Legal, RRHH y Finanzas. La base documental RAG se amplía con la documentación de esos departamentos. El objetivo de esta fase es validar que el control de acceso por departamento funciona correctamente con múltiples perfiles activos simultáneamente. Al final de la fase 2, todos los departamentos pueden usar el sistema.

Fase 3 — Despliegue a toda la organización (semana 9 en adelante). El sistema se abre a todos los empleados. Se activa el monitoreo operacional completo. Se establece el proceso de gestión de incidentes del propio sistema de IA (separado del agente de incidentes de TI, que es una funcionalidad, no la operación del sistema).

La infraestructura de cada componente

Conforme a RNF-04, TechCore v1.0 usa infraestructura de mercado accesible, no infraestructura propietaria.

Componente	Tecnología de mercado	Alternativa de contingencia
LLM principal	API de proveedor (ej. Anthropic, OpenAI)	Segundo proveedor configurado como fallback
Embeddings	API del mismo proveedor	Modelo de embeddings local (menor calidad)
Base vectorial	Servicio gestionado (ej. Pinecone, Weaviate)	PostgreSQL con extensión pgvector
Memoria persistente	Redis gestionado	PostgreSQL como KV store
Logging	Servicio de logs estructurados	Archivos locales + rotación
Orquestador	Servicio de aplicación (contenedor)	VM con auto-reinicio
Capa de presentación	Aplicación web + API REST	Bot de mensajería corporativa

La columna de alternativas de contingencia no es un plan B menor: es un plan B explícito y probado. Cada alternativa debe estar documentada y haber sido ejercitada en el entorno de desarrollo antes del lanzamiento.

Gestión de versiones de la base documental

La base documental RAG no es estática. Los runbooks se actualizan, las políticas cambian, los contratos se renuevan. TechCore necesita un proceso para mantener el índice vectorial sincronizado con la documentación real.

El proceso de actualización de la base documental tiene cuatro pasos:

Paso 1 — Detección de cambios. El sistema de gestión documental de TechCore (SharePoint, Confluence, o equivalente) tiene webhooks configurados que notifican al pipeline de ingestión cuando un documento es creado, modificado o eliminado.

Paso 2 — Re-ingestión del documento modificado. El pipeline de ingestión procesa el documento actualizado: lo fragmenta, genera los embeddings de cada fragmento, y los almacena en el índice vectorial con los metadatos actualizados.

Paso 3 — Eliminación de los fragmentos anteriores. Los fragmentos del documento anterior se eliminan del índice para evitar que el sistema recupere información desactualizada. La eliminación usa el identificador del documento como clave de búsqueda y eliminación masiva.

Paso 4 — Registro del cambio. El evento de actualización queda registrado en el log de administración con: nombre del documento, número de fragmentos anteriores eliminados, número de fragmentos nuevos indexados, y timestamp.

Este proceso asegura que el índice vectorial refleja siempre la versión más reciente de cada documento. No es necesario re-indexar toda la base documental cuando un documento cambia: solo se re-indexa el documento modificado.

El proceso de actualización de instrucciones del sistema

Las instrucciones del sistema se actualizan con mayor frecuencia que el código de la aplicación. Un cambio de política del departamento de TI puede requerir ajustar la instrucción del sistema del perfil TI sin tocar el código.

Para que ese proceso sea ágil y seguro, TechCore implementa un sistema de versionado de instrucciones:

```
Instrucciones del sistema — Estructura de versionado

/instrucciones/
  ti/
    v1.0.txt → versión en producción
    v1.1.txt → versión en prueba A/B (10% del tráfico TI)
    draft.txt → borrador sin desplegar
  legal/
    v1.0.txt → versión en producción
  rrhh/
    v1.0.txt → versión en producción
  finanzas/
    v1.0.txt → versión en producción
```

El orquestador lee la instrucción del sistema activa para cada perfil desde este almacén. Cambiar la instrucción del sistema en producción es una operación de escritura en el almacén, no un re-despliegue del servicio. El cambio tarda menos de un minuto en propagarse.

La funcionalidad de prueba A/B permite validar una nueva instrucción del sistema con un subconjunto del tráfico antes de activarla para todos los usuarios. Si las métricas de calidad del grupo de prueba son superiores al grupo de control, se promueve la nueva versión.

Mantenimiento operacional

El sistema en producción requiere cuatro actividades de mantenimiento recurrentes:

Revisión semanal de métricas. El responsable de operaciones revisa el panel de monitoreo y verifica que las nueve métricas operacionales están dentro de los objetivos. Si alguna métrica se acerca a su umbral de alerta, se abre un ticket de investigación.

Revisión mensual de la base documental. El equipo de cada departamento verifica que los documentos en el índice RAG están actualizados. Comparan la lista de documentos del índice contra el sistema documental corporativo y señalan cualquier discrepancia.

Revisión trimestral de instrucciones del sistema. El responsable de cada departamento revisa la instrucción del sistema de su perfil y verifica que sigue alineada con las políticas actuales del departamento. Si hay cambios relevantes, actualiza la instrucción y la somete a prueba A/B antes de promoverla.

Revisión anual de costos y proveedor. El equipo técnico revisa los costos de la API del LLM, la base vectorial y los servicios gestionados. Evalúa si la estructura de costos actual es sostenible y si existen alternativas que ofrezcan mejor relación costo-calidad para el volumen actual de uso.

Gestión de incidentes del propio sistema

TechCore tiene un agente de análisis de incidentes de TI, pero ese agente es una funcionalidad del sistema. El sistema mismo también puede tener incidentes: el orquestador puede quedar no disponible, la base vectorial puede degradarse, la API del LLM puede devolver errores.

El proceso de gestión de incidentes del sistema de IA sigue el mismo proceso que cualquier sistema de software corporativo: reporte en el sistema de tickets, clasificación por severidad, asignación al equipo responsable, y resolución documentada. La única diferencia es que el equipo responsable del sistema de IA necesita acceder a las trazas de interacción para diagnosticar qué falló y en qué etapa del pipeline.

Ese acceso está disponible a través del panel de monitoreo y de las consultas directas al sistema de logs. La traza de interacción completa, con latencias por etapa y errores registrados, es la herramienta de diagnóstico principal.

Con el diseño completo del sistema establecido en las secciones 03 a 08, el capítulo entra en su bloque de revisión crítica. La siguiente sección identifica los errores que emergen específicamente cuando se integran todos los componentes, distintos de los errores que ya aparecieron en capítulos individuales.

Capítulo 15 — Proyecto Integrador

Sección 09: Buenas prácticas y errores frecuentes en sistemas integrados

Los capítulos anteriores del módulo documentaron las buenas prácticas y anti-patrones de cada componente por separado: cómo estructurar instrucciones del sistema, cómo gestionar la ventana de contexto, cómo diseñar el pipeline RAG, cómo implementar herramientas seguras, cómo controlar agentes. No tiene sentido repetir esos catálogos aquí.

Esta sección se ocupa de algo diferente: los errores que solo se manifiestan cuando todos los componentes operan juntos. Son errores de integración, no de componente. Aparecen en producción, no en pruebas unitarias. Y son, por esa razón, los más costosos de detectar y corregir.

Error de integración 1: Conflicto entre memoria persistente e instrucciones del sistema

Descripción: El módulo de memoria persistente almacena preferencias del usuario. La instrucción del sistema del departamento establece el comportamiento esperado del asistente. Cuando ambas están en conflicto, el sistema puede comportarse de manera impredecible.

Ejemplo concreto: Un empleado de Legal establece en su perfil de memoria: "Responde siempre en inglés." La instrucción del sistema del departamento Legal dice: "Usa terminología jurídica en español." En una sesión, el LLM recibe ambas instrucciones en el contexto. El comportamiento resultante depende del orden en que aparecen las instrucciones y de cómo el LLM resuelve las contradicciones. Ese comportamiento no es predecible ni estable.

Solución: Establecer una jerarquía explícita de precedencia en el contexto: las instrucciones del sistema del departamento siempre prevalecen sobre las preferencias del usuario para los aspectos que afectan cumplimiento, seguridad o estilo institucional. Las preferencias del usuario prevalecen en aspectos de formato y conveniencia personal. La instrucción del sistema debe enunciar esta jerarquía explícitamente:

Las preferencias de formato del usuario (longitud de respuesta, uso de listas) son bienvenidas. Sin embargo, el idioma de las respuestas y la terminología legal son estándares del departamento y no se modifican por preferencias individuales.

Buena práctica: Definir antes del diseño qué aspectos del comportamiento son configurables por el usuario y cuáles son fijos por la institución. Documentar esa decisión en la instrucción del sistema.

Error de integración 2: Latencia acumulativa no anticipada

Descripción: Cada componente del pipeline tiene una latencia propia. En pruebas aisladas, cada componente es aceptablemente rápido. Pero la latencia total del sistema es la suma de todas las latencias, y esa suma puede exceder los objetivos cuando los componentes operan bajo carga real.

Ejemplo concreto: El módulo de memoria tarda 80 ms en recuperar el perfil del usuario. El motor RAG tarda 450 ms en procesar la consulta y devolver fragmentos. El LLM tarda 3.200 ms en generar la respuesta. El filtrado de salida tarda 120 ms. La latencia total es 3.850 ms: dentro del objetivo de 6 segundos. Pero si el motor RAG tiene un día con alta carga y tarda 1.800 ms, la latencia total sube a 5.200 ms, todavía dentro del objetivo. Y si además el LLM experimenta latencia alta de 5.500 ms, la latencia total llega a 7.500 ms, superando el umbral de alerta.

La latencia del peor caso de cada componente no es la suma de los promedios; es la suma de los percentiles 95 o 99 de cada uno. Ese cálculo generalmente revela que el sistema tiene margen mucho menor del que parece.

Solución: Presupuestar la latencia por componente con percentiles P95, no con promedios. El presupuesto total de latencia (por ejemplo, 8 segundos) se distribuye entre componentes con límites explícitos. Si un componente excede su límite de latencia, el orquestador debe tomar una decisión: degradar ese componente (servir sin memoria, sin RAG, con menos fragmentos) o devolver un error parcial al usuario.

Buena práctica: Implementar circuit breakers en cada componente. Si el módulo RAG supera 1 segundo de latencia tres veces consecutivas en un período, el orquestador sirve sin RAG y registra el degradado. El usuario recibe una respuesta basada solo en la instrucción del

sistema y la memoria, con una nota: "No pude acceder a la documentación interna en este momento."

Error de integración 3: Fragmentos RAG contaminando el razonamiento del agente

Descripción: El agente de análisis de incidentes usa RAG para recuperar runbooks relevantes. El pipeline RAG puede recuperar fragmentos de documentos de diferentes versiones del mismo runbook si el índice no está correctamente actualizado. El agente recibe fragmentos contradictorios y produce un diagnóstico incoherente.

Ejemplo concreto: El runbook de escalación fue actualizado en mayo de 2026: el nuevo umbral para escalar a P1 es de 45 minutos (antes era 60). Si el índice vectorial contiene fragmentos tanto de la versión anterior como de la nueva (porque la eliminación de fragmentos de la versión anterior no funcionó correctamente), el agente puede recuperar ambos fragmentos y generar una recomendación ambigua.

Solución: El proceso de actualización de la base documental (descrito en la sección 08) debe incluir una verificación post-actualización: consultar el índice para el documento recién actualizado y verificar que solo existen fragmentos de la nueva versión. Esa verificación puede automatizarse como parte del pipeline de ingestión.

Buena práctica: Incluir el número de versión y la fecha del documento en cada fragmento, y en la instrucción del agente indicar explícitamente que ante fragmentos contradictorios del mismo documento, debe usar el más reciente y reportar la inconsistencia.

Error de integración 4: Contexto de memoria que invalida el control de acceso

Descripción: El módulo de memoria puede almacenar referencias a información que el usuario obtuvo en una sesión anterior con acceso a documentos que ya no puede ver (porque su nivel de autorización cambió, o porque el documento fue reclasificado). Al inicio de la nueva sesión, el orquestador inyecta esa memoria en el contexto, introduciendo información que el usuario ya no debería tener.

Ejemplo concreto: Un empleado de RRHH con nivel de autorización elevado consulta datos salariales de un directivo. El agente almacena en la memoria: "El usuario revisó el contrato de X en la sesión anterior." Tres días después, ese empleado es degradado a nivel de

autorización estándar. En la siguiente sesión, la memoria inyectada en el contexto menciona el contrato de X, que el usuario ya no tiene permiso de ver.

Solución: El módulo de memoria no debe almacenar referencias a documentos confidenciales, solo referencias a documentos de clasificación `público_interno`. Las entradas de memoria que incluyen referencias a documentos deben pasar por el mismo filtro de control de acceso que el motor RAG, aplicado en el momento de recuperación (no en el momento de almacenamiento), usando el perfil de autorización actual del usuario.

Buena práctica: Al construir el contexto de memoria, aplicar el filtro de control de acceso sobre cada entrada antes de incluirla en el contexto. Si una entrada de memoria ya no es accesible para el usuario con su nivel de autorización actual, se omite silenciosamente.

Error de integración 5: El agente entra en bucle por herramienta que siempre falla

Descripción: El agente de incidentes usa herramientas en su ciclo ReAct. Si una herramienta devuelve siempre un error (por ejemplo, el sistema de tickets está en mantenimiento), el agente puede reintentar la llamada a la herramienta en cada iteración hasta agotar el límite de iteraciones, consumiendo tokens y tiempo sin producir valor.

Ejemplo concreto: El agente intenta verificar_solicitud(#4521) en la iteración 1. El sistema de tickets devuelve un error 503. El agente razona que necesita esa información y vuelve a intentar verificar_solicitud(#4521) en la iteración 2. Error 503. El agente vuelve a intentar. Ocho iteraciones, ocho errores, ningún progreso.

Solución: El módulo de herramientas implementa un registro de fallos por herramienta dentro del ciclo del agente. Si una herramienta falla dos veces consecutivas, el orquestador la marca como no disponible para el resto del ciclo y provee al agente un mensaje explícito: "La herramienta verificar_solicitud no está disponible. Continúa sin esa información o indica que no puedes completar el análisis."

Buena práctica: Toda herramienta expuesta a un agente debe tener un mecanismo de degradación explícita. El agente debe ser capaz de producir un resultado parcial útil cuando una herramienta no está disponible, en lugar de fallar completamente.

La tabla de errores de integración

Error de integración	Componentes involucrados	Señal de detección	Mitigación
Conflicto memoria-instrucciones	Memoria + instrucción del sistema	Comportamiento inconsistente	Jerarquía de precedencia explícita
Latencia acumulativa	Todos los componentes del pipeline	P95 latencia > umbral	Circuit breakers + degradación
Fragmentos RAG contradictorios	RAG + agente	Diagnóstico ambiguo	Verificación post-actualización
Memoria con acceso inválido	Memoria + control de acceso	Información no autorizada	Filtro de acceso en recuperación
Bucle en herramienta fallida	Agente + herramientas	Iteraciones sin progreso	Registro de fallos + herramienta desactivada

La revisión de esta tabla debe formar parte del proceso de revisión técnica de cualquier sistema de IA que integre los cinco componentes involucrados.

Con los errores de integración identificados, la siguiente sección presenta el caso completo de implementación de TechCore como referencia consolidada.

Capítulo 15 — Proyecto Integrador

Sección 10: Caso completo de implementación

Esta sección presenta el caso TechCore como una unidad completa: el ciclo de vida de una interacción real, desde que el usuario escribe su mensaje hasta que recibe la respuesta, con todos los componentes del sistema en operación. No es un resumen de las secciones anteriores: es el escenario de integración que verifica que el diseño produce el comportamiento esperado.

El escenario

Usuario: Marcos Velarde, analista de soporte TI, departamento TI, nivel de autorización estándar.

Horario: Martes 08:42, turno matutino.

Memoria persistente disponible: Preferencia por respuestas en viñetas. Ticket #4521 activo (servidor web-03 caído, abierto hace 3 días).

Mensaje del usuario:

"Buenos días. Necesito ayuda con el web-03. Sigue sin responder y el ticket lleva 3 días abierto. ¿Esto ya debería haberse escalado?"

Paso 1: El orquestador recibe la entrada

El orquestador registra la entrada del usuario con timestamp y genera el `trace_id` de la interacción:

```
trace_id: trc-20260725-0842-emp0472
user_id: emp-0472 (Marcos Velarde)
department: ti
timestamp: 2026-07-25T08:42:17Z
input: "Buenos días. Necesito ayuda con el web-03..."
```

Paso 2: Recuperación del perfil y la instrucción del sistema

El orquestador consulta el módulo de instrucciones para el perfil TI. Recupera la instrucción del sistema v1.0. El tamaño de la instrucción es de 987 tokens.

Paso 3: Recuperación de memoria persistente

El orquestador consulta el KV store con la clave `emp-0472`. Recupera:

```
{
  "preferencia_formato": "viñetas cortas",
  "rol": "analista de soporte, turno matutino",
  "tickets_activos": ["#4521"],
  "ultima_sesion": {
    "fecha": "2026-07-22",
    "tema": "runbook de escalación de incidentes P1"
  }
}
```

El módulo de memoria formatea esta información en 412 tokens para la Zona 2 del contexto.

Paso 4: Recuperación RAG

El orquestador detecta que el mensaje contiene referencias a escalación y a un incidente activo. Ejecuta una búsqueda RAG con la consulta: "escalación incidente servidor web tiempo de resolución política".

El motor RAG devuelve 5 fragmentos candidatos. El filtro de control de acceso verifica que todos pertenecen al departamento TI (el usuario tiene acceso). Los 5 fragmentos pasan el filtro. Se incluyen 4 de los 5 (el quinto es redundante con el primero) en 1.843 tokens para la Zona 3.

Los fragmentos recuperados provienen de:

- `Runbook-Incidentes-TI-v4.2.pdf`, sección 3.2 (escalación a soporte nivel 2)
- `Política-Gestión-Incidentes-v2.1.pdf`, sección 5 (tiempos de escalación)
- `Runbook-Incidentes-TI-v4.2.pdf`, sección 4.1 (clasificación P1)
- `Acuerdo-Nivel-Servicio-TechCore-v3.0.pdf`, sección 2 (SLA para servidores web)

Paso 5: Construcción del contexto completo

El orquestador ensambla el contexto de 4.344 tokens:

```
[ZONA 1 — INSTRUCCIÓN DEL SISTEMA — 987 tokens]
Eres el asistente interno de TI de TechCore...

[ZONA 2 — CONTEXTO DEL USUARIO — 412 tokens]
[CONTEXTO DEL USUARIO - emp-0472]
Departamento: TI | Rol: Analista de soporte, turno matutino
Preferencias: viñetas cortas
Solicitudes activas: Ticket #4521 (web-03 caído, abierto hace 3 días)
Última sesión (22/07): consultó runbook de escalación P1

[ZONA 3 — DOCUMENTACIÓN RECUPERADA — 1843 tokens]
[DOCUMENTACIÓN INTERNA RECUPERADA]
Fuente 1: Runbook-Incidentes-TI-v4.2.pdf — Sección 3.2...
Fuente 2: Política-Gestión-Incidentes-v2.1.pdf — Sección 5...
...

[ZONA 4 — HISTORIAL DE CONVERSACIÓN — 1102 tokens]
(turno actual: mensaje de Marcos)
```

Paso 6: Inferencia del LLM

El contexto se envía al LLM. El LLM razona sobre el mensaje considerando:

- El contexto de memoria (Marcos tiene el ticket #4521 activo hace 3 días)
- La política de escalación recuperada (P2 sin resolución en 45 min → P1; en este caso, 72 horas es claramente un caso que debió escalar)
- El SLA para servidores web (el Acuerdo de Nivel de Servicio establece que un servidor web en producción tiene SLA de resolución de 4 horas para incidentes P2)

El LLM no solicita ninguna herramienta en este turno: puede responder completamente con la información del contexto. Genera una respuesta de 287 tokens.

Paso 7: Filtrado de salida

La respuesta del LLM pasa por el filtro de salida. El filtro verifica:

- No contiene datos personales sensibles: ninguno detectado.

- Las citas de documentos están en los fragmentos recuperados: verificado.
- No hay señales de inyección de prompt en la salida: ninguna detectada.

La respuesta pasa el filtro.

Paso 8: Respuesta al usuario

El sistema entrega la respuesta a Marcos a las 08:42:22 (5 segundos desde la entrada):

Asistente:

Buenas, Marcos. Sí, ese ticket debió escalarse hace tiempo.

¿Por qué debió escalarse?

- La Política de Gestión de Incidentes (sección 5) establece que un incidente P2 sin resolución en 45 minutos debe reclasificarse a P1.
- El ticket #4521 lleva 72 horas abierto sin resolución: excede ampliamente ese umbral.
- El SLA para servidores web de producción establece resolución de incidentes P2 en máximo 4 horas.

Lo que recomiendo ahora:

- Reclasificar el ticket #4521 de P2 a P1 inmediatamente.
- Notificar al Gerente de TI (según política, en los 15 minutos siguientes a la reclasificación).
- Verificar si hay logs de error recientes en web-03 — el 28 de junio tuviste un incidente similar de disco lleno resuelto con limpieza de logs.

¿Quieres que cree la actualización del ticket con la reclasificación a P1?

Paso 9: Actualización de memoria

El orquestador actualiza la memoria persistente de Marcos con la nueva información relevante de la sesión:


```
{
  "key": "ultima_sesion",
  "value": {
    "fecha": "2026-07-25",
    "tema": "escalación ticket #4521 a P1, servidor web-03"
  }
}
```

Paso 10: Registro completo de la traza

El sistema registra la traza completa de la interacción con todos los campos definidos en la sección 07. El registro es inmutable.

Lo que este escenario demuestra

Esta interacción de cinco segundos integró seis de los siete componentes de la arquitectura:

Componente	Contribución
Instrucción del sistema	Definió el rol y las restricciones del asistente para TI
Memoria persistente	Proveyó el contexto del ticket activo y las preferencias
Motor RAG	Recuperó la política de escalación y el SLA aplicable
Orquestador	Coordinó todos los componentes y construyó el contexto
LLM	Razonó sobre el contexto y generó la respuesta
Observabilidad	Registró la traza completa con latencias por etapa

El único componente que no se activó en este turno fue el módulo de herramientas, que hubiera entrado en el siguiente turno si Marcos hubiera respondido "Sí" a la propuesta de reclasificar el ticket.

Esta interacción es representativa del 80 % de las consultas que TechCore recibirá: preguntas de procedimiento que combinan recuperación de documentos con contexto de memoria. El 20 % restante activará el módulo de herramientas o el agente de incidentes para casos más complejos.

Con el caso completo de implementación como referencia, la siguiente sección propone el laboratorio integrador en tres niveles de profundidad.

Capítulo 15 — Proyecto Integrador

Sección 11: Laboratorio integrador

El laboratorio integrador tiene un único objetivo: que hagas el trabajo de diseñar, no de describir lo que harías. Está estructurado en tres niveles de profundidad. Puedes completar solo el nivel básico, los dos primeros, o los tres. Cada nivel tiene valor independiente. No se requiere infraestructura de producción para ninguno de los tres niveles: solo papel, diagrama y capacidad de razonamiento.

Nivel básico — Diseñar la arquitectura para un caso propio

Consigna:

Elige un problema de negocio de tu propio contexto (laboral, académico o personal) que cumpla estas tres condiciones:

1. Existe documentación interna relevante que podría ser indexada en RAG.
2. Hay al menos una acción sobre un sistema externo que el asistente debería poder ejecutar.
3. El asistente debe recordar algo entre sesiones para ser verdaderamente útil.

Si no encuentras un caso en tu contexto, usa este: **Asistente de atención al cliente para una ferretería con catálogo de 3.000 productos, precios que cambian semanalmente, y un sistema de inventario al que se puede consultar el stock en tiempo real.**

Entregables:

1. **Descripción del problema** (máximo 200 palabras): qué problema resuelve el asistente, quiénes son los usuarios, qué decisión tomarían mejor o más rápido con el asistente que sin él.
2. **Diagrama de arquitectura** (puede ser dibujado a mano o con cualquier herramienta de diagramas): los siete componentes de la arquitectura de TechCore adaptados a tu caso. Nombra cada componente con su función específica en tu caso, no con nombres genéricos.

3. **Tabla de cobertura de componentes:** misma tabla que se usó en la sección 02 para TechCore, pero para tu caso. Verifica que cada componente del módulo (instrucciones del sistema, memoria, RAG, herramientas, agentes, observabilidad, seguridad) está activo en tu diseño. Si algún componente no aplica, justifica por qué.

Criterio de calidad: El diagrama debe poder ser entendido por alguien que no estuvo en el proceso de diseño. Si tienes que explicar verbalmente cada caja, el diagrama no es suficientemente claro.

Nivel intermedio — Documentar las decisiones de diseño

Consigna:

Toma el diseño producido en el nivel básico y documenta las cinco decisiones de arquitectura más importantes. Para cada decisión:

1. **La decisión:** qué elegiste hacer (una frase).
2. **La alternativa considerada:** qué otra opción evaluaste (una frase).
3. **La justificación:** por qué elegiste la opción que elegiste y no la alternativa (dos a cuatro oraciones).
4. **El costo de la decisión:** qué sacrificaste al tomar esa decisión (una o dos oraciones).
5. **La condición de reversión:** en qué circunstancia cambiarías esta decisión (una oración).

Plantilla de decisión:

DECISIÓN [número]: [título breve]

Elegí: [opción elegida]

Alternativa: [opción descartada]

Justificación:

[Por qué la opción elegida es mejor que la alternativa en este contexto específico]

Costo:

[Qué se sacrifica al tomar esta decisión]

Condición de reversión:

[Cuándo esta decisión debería revisarse]

Cinco preguntas guía para identificar las decisiones de arquitectura de tu caso:

- ¿Por qué RAG en lugar de fine-tuning, o viceversa?
- ¿Por qué un agente en lugar de un flujo fijo de herramientas, o viceversa?
- ¿Por qué ventana deslizante en lugar de resumen incremental para el historial, o viceversa?
- ¿Por qué confirmación de usuario antes de ejecutar herramientas, o sin confirmación?
- ¿Por qué un único perfil de instrucciones del sistema, o perfiles diferenciados por rol?

Si tu caso tiene decisiones más relevantes que las sugeridas, documéntalas en lugar de las que no apliquen.

Criterio de calidad: Una decisión bien documentada debe permitir a otro ingeniero entender por qué se tomó esa decisión en este contexto y qué haría que cambiara. Si la justificación es genérica ("es más escalable", "es la mejor práctica"), no es suficientemente buena. La justificación debe ser específica para tu caso.

Nivel avanzado — Análisis de riesgos y mitigaciones

Consigna:

Identifica tres riesgos del diseño que produciste. Un riesgo no es un error de implementación: es una condición del entorno o del uso del sistema que, si ocurre, puede degradar o comprometer el sistema. Para cada riesgo:

1. **Descripción del riesgo:** qué puede salir mal, en qué circunstancias (dos a tres oraciones).
2. **Probabilidad estimada:** alta / media / baja, con una justificación de la estimación.
3. **Impacto estimado:** crítico / alto / medio / bajo, con una justificación.
4. **Señal de detección:** cómo sabrías que el riesgo se está materializando antes de que cause daño severo.
5. **Mitigación propuesta:** qué cambio de diseño o qué proceso operacional reduce el riesgo a un nivel aceptable.

Plantilla de riesgo:

```
RIESGO [número]: [nombre del riesgo]
```

```
Descripción:
```

```
[Qué puede salir mal y en qué circunstancias]
```

```
Probabilidad: [alta/media/baja]
```

```
Justificación: [por qué estimas esa probabilidad]
```

```
Impacto: [crítico/alto/medio/bajo]
```

```
Justificación: [por qué estimas ese impacto]
```

```
Señal de detección:
```

```
[Indicador observable que permite detectar el riesgo antes de que escale]
```

```
Mitigación propuesta:
```

```
[Cambio de diseño o proceso que reduce el riesgo a nivel aceptable]
```

Tres categorías de riesgo para guiar la búsqueda:

- **Riesgos de datos:** la base documental RAG se desactualiza, los documentos tienen errores, la calidad del chunking es deficiente para ciertos tipos de documentos.

- **Riesgos de comportamiento del modelo:** el LLM alucina con documentos de dominio especializado, la instrucción del sistema se vuelve ambigua a medida que se agregan casos borde, el agente entra en bucles en escenarios no anticipados.
- **Riesgos operacionales:** el costo de la API del LLM escala inesperadamente, la disponibilidad del proveedor de LLM cae, un cambio en la API del proveedor rompe el orquestador.

Criterio de calidad: La mitigación debe ser concreta. "Mejorar el monitoreo" no es una mitigación concreta. "Agregar una alerta cuando el número de fragmentos recuperados con cita cae por debajo del 50 % durante tres días consecutivos, lo que indica posible desactualización del índice RAG" es una mitigación concreta.

Reflexión final del laboratorio

El laboratorio tiene tres niveles porque integrar un sistema de IA requiere tres capacidades distintas:

El nivel básico ejercita la síntesis estructural: ¿puedo producir una arquitectura coherente que integre todos los componentes necesarios para resolver el problema?

El nivel intermedio ejercita el razonamiento de diseño: ¿puedo justificar por qué elegí esta arquitectura y no otra, con argumentos específicos para este caso?

El nivel avanzado ejercita el pensamiento de riesgo: ¿puedo anticipar qué puede fallar en un sistema que diseñé, antes de que falle en producción?

Las tres capacidades son necesarias para un AI Engineer profesional. Un sistema diseñado sin razonamiento documentado no puede mantenerse. Un sistema diseñado sin análisis de riesgos sorprende a quien lo opera en el momento más inoportuno.

Con el laboratorio completo, el capítulo entra en su bloque de cierre. La siguiente sección entrega el checklist profesional definitivo del módulo.

Capítulo 15 — Proyecto Integrador

Sección 12: Checklist final del AI Engineer

Este checklist es el instrumento de revisión definitivo del módulo. Sintetiza los criterios de calidad de todas las dimensiones del Context Engineering en un único documento aplicable a cualquier proyecto propio. No es un resumen de los checklists parciales de capítulos anteriores: cubre exclusivamente los criterios que aplican a un sistema integrado completo.

Usa este checklist antes de pasar cualquier sistema de IA basado en Context Engineering a un entorno de producción. Las preguntas que no puedas responder con "sí" son los aspectos del diseño que necesitan trabajo adicional.

BLOQUE 1 — Instrucciones del sistema y comportamiento base (5 ítems)

- [] **1.1** La instrucción del sistema define explícitamente qué puede hacer el asistente, qué no puede hacer, y cómo debe responder en los casos borde más frecuentes.
 - [] **1.2** Si el sistema tiene múltiples perfiles de usuario (por departamento, rol o tipo de acceso), cada perfil tiene su propia instrucción del sistema con diferencias justificadas por los requisitos del caso de uso, no por preferencias arbitrarias.
 - [] **1.3** Las instrucciones del sistema se gestionan como datos (fuera del código fuente), pueden actualizarse sin re-desplegar el servicio, y tienen un historial de versiones con fecha y responsable de cada cambio.
 - [] **1.4** El sistema tiene un proceso de prueba para nuevas versiones de instrucciones del sistema antes de activarlas para todos los usuarios (prueba A/B, canario, o equivalente).
 - [] **1.5** La instrucción del sistema declara explícitamente la jerarquía de precedencia entre las reglas institucionales y las preferencias del usuario, para los aspectos en que puedan entrar en conflicto.
-

BLOQUE 2 — Gestión del contexto y la ventana (4 ítems)

- [] **2.1** El presupuesto de tokens por turno está definido explícitamente y distribuido entre las zonas del contexto (instrucción del sistema, memoria, RAG, historial de conversación). El sistema no envía contextos que superan el presupuesto definido.
 - [] **2.2** La estrategia de gestión del historial de conversación (ventana deslizante, resumen incremental, u otra) está documentada. El sistema degrada correctamente cuando el historial supera la capacidad de la ventana: no trunca silenciosamente sin informar al LLM del límite.
 - [] **2.3** El sistema puede manejar correctamente el caso borde en que la combinación de instrucción del sistema + memoria + RAG + historial supera el presupuesto de tokens: sabe qué zona priorizar y qué zona comprimir cuando el presupuesto se agota.
 - [] **2.4** El costo estimado de tokens por interacción (promedio y P95) ha sido calculado con datos reales o estimaciones basadas en el tipo de consulta esperada, y es sostenible dentro del presupuesto de la API.
-

BLOQUE 3 — Memoria persistente (4 ítems)

- [] **3.1** El sistema distingue claramente entre qué información merece ser memorizada entre sesiones y qué no. El criterio de selección está documentado y se aplica consistentemente, no discrecionalmente.
 - [] **3.2** La memoria persistente tiene TTL (tiempo de expiración) configurado para cada tipo de información, y el sistema elimina automáticamente las entradas caducadas.
 - [] **3.3** El usuario puede solicitar la eliminación de su memoria persistente y el sistema lo honra efectivamente: borra las entradas del almacén y no las recupera en sesiones posteriores.
 - [] **3.4** La memoria persistente pasa por el mismo filtro de control de acceso que el motor RAG: las entradas que hacen referencia a información a la que el usuario ya no tiene acceso se omiten al construir el contexto.
-

BLOQUE 4 — Recuperación aumentada (RAG) (6 ítems)

- [] **4.1** El tamaño de los chunks y el solapamiento entre chunks han sido calibrados para el tipo de documentos que se indexan. El tamaño por defecto (400 tokens con solapamiento

de 80) es un punto de partida, no un valor definitivo para todos los casos.

- [] **4.2** Cada fragmento del índice vectorial tiene metadatos de origen completos: nombre del documento, sección, fecha de última modificación, propietario y nivel de clasificación.
- [] **4.3** El proceso de actualización del índice cuando un documento cambia está automatizado y verificado: el sistema elimina los fragmentos de la versión anterior antes de indexar la versión nueva, y verifica que la eliminación fue exitosa.
- [] **4.4** El control de acceso se aplica en el nivel del recuperador (antes de que los fragmentos lleguen al contexto del LLM), no solo en el nivel de la salida del LLM.
- [] **4.5** El sistema puede responder correctamente cuando no encuentra información relevante en el índice: no inventa respuestas, indica explícitamente que no encontró documentación aplicable y sugiere una alternativa (persona de contacto, sistema alternativo).
- [] **4.6** Las respuestas del sistema que usan información de documentos incluyen siempre la referencia al documento fuente (nombre, sección, fecha). El usuario puede verificar la fuente sin esfuerzo adicional.

BLOQUE 5 — Herramientas y MCP (4 ítems)

- [] **5.1** Cada herramienta expuesta al LLM tiene una especificación completa: nombre, descripción, parámetros con tipos, respuesta esperada, permisos requeridos y comportamiento ante fallo.
- [] **5.2** Las herramientas que producen efectos en sistemas externos (creación, modificación, eliminación, notificación) requieren confirmación explícita del usuario antes de ejecutarse, excepto cuando el caso de uso justifica la ejecución automática y esa decisión está documentada.
- [] **5.3** El sistema tiene comportamiento de degradación controlada para cada herramienta: cuando una herramienta falla, el usuario recibe una respuesta útil (no un error técnico) y una alternativa de acción.
- [] **5.4** El acceso a herramientas está controlado por el perfil del usuario: las herramientas que requieren permisos específicos no aparecen como disponibles para usuarios sin esos permisos (no solo fallan cuando se intentan usar).

BLOQUE 6 — Agentes (4 ítems, solo si el sistema incluye agentes)

- [] **6.1** La decisión de usar un agente en lugar de un flujo fijo de herramientas está justificada: el caso de uso requiere condicionalidad en la secuencia de pasos que no puede definirse antes de ejecutar.
 - [] **6.2** El agente tiene un límite máximo de iteraciones configurado. Cuando alcanza ese límite sin converger en una respuesta, presenta un resumen de lo que encontró y escala a un humano, en lugar de fallar con un error técnico.
 - [] **6.3** La instrucción del sistema del agente define explícitamente el proceso de trabajo que debe seguir (pasos, criterios de decisión, condiciones de parada), no solo las capacidades que tiene.
 - [] **6.4** Las herramientas del agente tienen mecanismos de detección de fallos recurrentes: si una herramienta falla dos veces consecutivas, el agente la marca como no disponible y continúa el análisis sin ella o informa que no puede completar la tarea sin esa información.
-

BLOQUE 7 — Observabilidad (5 ítems)

- [] **7.1** Cada interacción genera una traza completa: entrada del usuario, contexto enviado al LLM (con tokens por zona), respuesta del LLM, herramientas invocadas, fuentes RAG recuperadas, y latencias por etapa.
 - [] **7.2** Las trazas son inmutables y tienen un período de retención definido que cumple los requisitos de auditoría del caso de uso.
 - [] **7.3** El sistema monitorea latencia (P50 y P95 por etapa), tasa de error por componente, y costo de tokens por usuario/departamento, con umbrales de alerta configurados.
 - [] **7.4** El equipo de operaciones puede diagnosticar un incidente del sistema a partir de las trazas almacenadas, sin necesidad de reproducir la interacción problemática.
 - [] **7.5** Los cambios en instrucciones del sistema, en el índice RAG y en la configuración de herramientas quedan registrados en un log de administración separado de las trazas de interacción.
-

BLOQUE 8 — Seguridad (5 ítems)

- [] **8.1** El control de acceso opera en el nivel de recuperación: los documentos a los que el usuario no tiene acceso nunca llegan al contexto del LLM, no solo son filtrados en la salida.
 - [] **8.2** El sistema tiene un mecanismo de detección de prompt injection sobre la entrada del usuario, con respuesta de bloqueo y registro del intento ante detección positiva.
 - [] **8.3** El filtro de salida verifica que las respuestas del LLM no contienen: datos personales sensibles fuera del alcance del usuario, referencias a documentos no recuperados en ese turno, ni instrucciones que sugieran manipulación exitosa del sistema.
 - [] **8.4** El equipo tiene un proceso documentado de respuesta ante un incidente de seguridad del sistema de IA (acceso no autorizado a documentos, exposición de datos, prompt injection exitosa).
 - [] **8.5** Los documentos de alta clasificación (restringido) generan alertas de auditoría en tiempo real al responsable de cumplimiento cuando son accedidos, además de quedar en la traza general.
-

BLOQUE 9 — Operación y evolución (3 ítems)

- [] **9.1** La estrategia de despliegue es incremental: el sistema fue validado con un subconjunto de usuarios antes de operar con toda la organización.
 - [] **9.2** El proceso de actualización de la base documental RAG está automatizado o tiene un calendario explícito de revisión manual, y el responsable de mantenerla actualizada está identificado por departamento.
 - [] **9.3** El equipo tiene un proceso de revisión periódica de las instrucciones del sistema por parte del responsable de negocio de cada departamento, con una frecuencia adecuada al ritmo de cambio de las políticas del caso de uso.
-

Cómo usar este checklist en producción

Este checklist tiene 40 ítems. No todos son bloqueantes para el lanzamiento inicial: algunos son críticos (control de acceso, trazas de auditoría, comportamiento ante herramientas fallidas) y otros son mejoras que pueden incorporarse después del lanzamiento (TTL de memoria, pruebas A/B de instrucciones).

Antes de llevar cualquier sistema de IA a producción, clasifica los ítems no cumplidos en dos categorías:

Bloqueantes: ítems sin los cuales el sistema expone datos de forma incorrecta, no puede auditarse, o tiene comportamientos impredecibles ante fallos. Los bloqueantes deben cumplirse antes del lanzamiento.

Post-lanzamiento: ítems que mejoran la calidad, la operabilidad o la experiencia del usuario pero cuya ausencia no compromete la seguridad ni la confiabilidad básica del sistema. Estos pueden incorporarse en las primeras semanas de operación.

El criterio de clasificación es tuyo, no de este libro. Conoces tu caso de uso, tu organización y tus usuarios. Lo que es post-lanzamiento para un chatbot interno de bajo riesgo puede ser bloqueante para un sistema que maneja datos regulados.

Con el checklist completo, las dos secciones finales del capítulo cierran el módulo.

Capítulo 15 — Proyecto Integrador

Sección 13: Resumen del módulo

El Módulo 3 cubrió Context Engineering Profesional a lo largo de doce capítulos y un proyecto integrador. No tiene sentido recapitular el contenido de cada capítulo: los resúmenes parciales ya están en cada uno de ellos. Lo que sí tiene sentido es enunciar los principios fundamentales que atraviesan todo el módulo.

Estos son los seis principios que un AI Engineer profesional debe llevarse del Módulo 3.

Principio 1: El contexto es el producto

El LLM no es el sistema. El LLM es el motor. El sistema es todo lo que ocurre alrededor del LLM: cómo se construye el contexto, qué información se recupera, cómo se gestiona la memoria, qué herramientas se exponen, cómo se controla el comportamiento del modelo, cómo se registra y monitorea cada interacción.

La calidad de un sistema de IA en producción está determinada principalmente por la calidad del Context Engineering que lo sostiene, no por la versión del modelo que usa. Un modelo mediano con excelente Context Engineering supera consistentemente a un modelo de última generación con Context Engineering deficiente.

Esta es la competencia central del módulo: construir el entorno que permite a un LLM comportarse de manera predecible, útil y segura en un contexto específico.

Principio 2: Diseña para la ventana, no para el modelo

Cada interacción con un LLM tiene un presupuesto de tokens. Ese presupuesto es el recurso más escaso del sistema. La ingeniería de contexto es, en gran medida, la práctica de administrar ese presupuesto con criterio: qué incluir, qué comprimir, qué excluir, y en qué orden.

Las cuatro zonas del contexto —instrucción del sistema, memoria, recuperación de documentos, historial de conversación— compiten por el mismo espacio. Las decisiones sobre cómo distribuir ese espacio tienen consecuencias directas en la calidad de las respuestas, la latencia del sistema y el costo de operación.

Un AI Engineer que no tiene claridad sobre cómo está usando el presupuesto de tokens de su sistema no tiene control real sobre su comportamiento.

Principio 3: La memoria es intencional, no acumulativa

La memoria persistente de un sistema de IA no es un archivo de todo lo que ocurre. Es una selección intencional de la información que mejora las interacciones futuras. La distinción es importante porque un sistema que memoriza indiscriminadamente no solo consume espacio de almacenamiento: degrada la calidad del contexto llenándolo de información irrelevante, y introduce riesgos de privacidad al conservar datos que el usuario no espera que el sistema recuerde.

El criterio correcto para memorizar no es "esto podría ser útil algún día". Es "esto cambiará el comportamiento del sistema en la próxima sesión de manera que el usuario considere valiosa".

Principio 4: RAG provee conocimiento, las herramientas proveen agencia

RAG y herramientas son complementarios, no intercambiables. RAG permite al sistema acceder a información que no está en los parámetros del modelo: documentación interna, bases de conocimiento específicas del dominio, información actualizada. Las herramientas permiten al sistema actuar sobre el mundo externo: crear registros, consultar estados en tiempo real, notificar a personas.

Un sistema que solo tiene RAG puede responder preguntas sobre la realidad. Un sistema que solo tiene herramientas puede actuar pero sin conocimiento. Un sistema con ambos puede razonar sobre el conocimiento disponible y actuar en consecuencia. Esa combinación es la que habilita los casos de uso de mayor valor.

Principio 5: La complejidad del agente se justifica con la complejidad de la tarea

Los agentes son la herramienta más poderosa del Context Engineering y también la más costosa en términos de complejidad de diseño, depuración y operación. La decisión de usar un agente en lugar de un flujo fijo de herramientas debe estar justificada por una propiedad específica de la tarea: la secuencia de pasos no puede definirse antes de ejecutarla.

Cuando esa condición no aplica —cuando la tarea es un flujo determinístico de pasos conocidos— un orquestador simple es más confiable, más barato, más fácil de depurar y más fácil de mantener que un agente. La complejidad adicional del agente no se justifica solo porque la tecnología esté disponible.

Principio 6: Observabilidad y seguridad son dimensiones del diseño, no capas adicionales

Un sistema de IA que no tiene observabilidad desde el primer día opera a ciegas. No puedes mejorar lo que no puedes medir. No puedes diagnosticar lo que no puedes ver. No puedes auditar lo que no registraste. Instrumentar la observabilidad después del lanzamiento es más costoso y más incompleto que diseñarla desde el primer diagrama de arquitectura.

Lo mismo aplica a la seguridad. Un control de acceso que opera en el nivel de la salida del LLM es una segunda línea de defensa, no la primera. La primera línea es que información no autorizada nunca llegue al contexto del LLM. Un sistema seguro por diseño es más robusto que un sistema asegurado por capas de filtros sobre un diseño fundamentalmente abierto.

El mapa del módulo

Capítulo	Componente	Principio central que desarrolla
04	Ventana de contexto	Principio 2 — diseña para la ventana
05	Instrucciones del sistema	Principio 1 — el contexto es el producto
06	Memoria (sesión)	Principio 3 — la memoria es intencional
07	Memoria (persistente)	Principio 3 — la memoria es intencional
08	RAG — fundamentos	Principio 4 — RAG provee conocimiento
09	RAG — avanzado	Principio 4 — RAG provee conocimiento
10	Herramientas y MCP	Principio 4 — las herramientas proveen agencia
11	Agentes	Principio 5 — la complejidad se justifica
12	Arquitecturas complejas	Principios 4 y 5 combinados
13	Observabilidad	Principio 6 — observabilidad desde el diseño
14	Seguridad	Principio 6 — seguridad desde el diseño
15	Proyecto integrador	Los seis principios en acción simultánea

El Módulo 3 te da los instrumentos para construir sistemas de IA que funcionan en producción con el rigor de un sistema de software profesional. El Módulo 4 construye sobre esa base para abordar las arquitecturas completas de los sistemas de IA modernos: cómo se componen múltiples agentes, cómo se escalan los sistemas de IA a millones de usuarios, y cómo se diseñan las arquitecturas de infraestructura que sostienen esos sistemas.

Los principios del Módulo 3 no desaparecen en el Módulo 4: son los bloques de construcción de esas arquitecturas más complejas.

Capítulo 15 — Proyecto Integrador

Sección 14: Evaluación final

Esta evaluación está diseñada para que puedas aplicarla sin un instructor. No mide si recuerdas definiciones: mide si puedes aplicar los conceptos del módulo para resolver problemas nuevos. La escala de evaluación es simple: si puedes responder cada pregunta sin consultar el material, completaste el módulo. Si necesitas releer una sección para responder, esa sección merece una segunda lectura.

Dedica entre 60 y 90 minutos a esta evaluación. Responde en papel o en un documento separado antes de revisar las notas de contraste al final.

Parte A — Preguntas de aplicación directa

Estas preguntas tienen respuestas verificables. No son preguntas de opinión.

A1. Un sistema de IA para atención al cliente recibe esta consulta de un usuario: "¿Cuánto tiempo tardan los envíos a las Islas Canarias?" El sistema tiene un índice RAG con el catálogo de productos y las políticas de envío. El motor RAG devuelve tres fragmentos: uno de la política de envíos peninsulares (no aplica), uno del FAQ de Canarias con los tiempos correctos, y uno de una actualización de precios de 2024 (no aplica). El LLM incluye información del fragmento de precios de 2024 en su respuesta porque encuentra una mención a "Canarias" en ese fragmento.

¿Qué falla en el sistema y en qué etapa? ¿Qué cambiarías en el diseño para evitar este tipo de error?

A2. Un agente de análisis de reclamos de seguros tiene acceso a cuatro herramientas: `consultar_poliza`, `verificar_historial_siniestros`, `calcular_indemnizacion`, y `aprobar_pago`. El agente procesa un reclamo y propone aprobar el pago automáticamente sin mostrar el cálculo al operador.

¿Qué principio de diseño de agentes viola este comportamiento? ¿Cómo modificarías la instrucción del sistema del agente para corregirlo?

A3. Un sistema de asistente médico interno para enfermeros tiene una instrucción del sistema que incluye: "Proporciona información clínica detallada sobre medicamentos, incluyendo dosis y contraindicaciones." Un enfermero con credenciales válidas pregunta sobre la dosis máxima de un medicamento controlado. El sistema responde correctamente. Un estudiante de enfermería que tiene acceso al sistema (para consultas de aprendizaje) hace la misma pregunta.

¿Qué mecanismo de diseño del módulo aplica directamente a este caso? ¿Qué cambios concretos implementarías?

A4. Un equipo construyó un sistema de IA con las siguientes características:

- El historial de conversación se conserva completo, sin límite.
- La instrucción del sistema ocupa 3.000 tokens.
- El sistema no usa RAG.
- El modelo tiene una ventana de 8.000 tokens.

Después de 15 turnos de conversación, el sistema empieza a "olvidar" instrucciones del sistema y su comportamiento se vuelve inconsistente.

Explica exactamente por qué ocurre esto y propón dos estrategias para resolverlo sin cambiar el modelo.

A5. Un sistema de asistente empresarial tiene un módulo de memoria persistente que almacena todo lo que el usuario menciona en sus conversaciones. Un usuario menciona, de pasada: "Esta semana estoy de mal humor porque mi jefe me llamó la atención." El sistema almacena ese fragmento como memoria.

Identifica al menos dos problemas que ese comportamiento puede causar y explica cómo el diseño correcto del módulo de memoria los evita.

Parte B — Preguntas de diseño

Estas preguntas no tienen una única respuesta correcta. Se evalúan por la solidez del razonamiento.

B1. Una startup de legaltech quiere construir un asistente que ayude a abogados a revisar contratos. El asistente debe identificar cláusulas problemáticas según la ley argentina, comparar el contrato con plantillas estándar del estudio, y sugerir modificaciones. Los contratos son confidenciales y no deben mezclarse entre clientes.

Define la arquitectura en términos de los siete componentes del módulo. Para cada componente, explica su función específica en este caso. Identifica cuál es la decisión de diseño más crítica de esta arquitectura y justifica por qué.

B2. Un sistema de asistente de soporte técnico fue desplegado hace seis meses. En las últimas dos semanas, el equipo nota que las respuestas sobre un producto específico son incorrectas: el producto tuvo una actualización importante y el asistente sigue respondiendo sobre la versión anterior.

Describe paso a paso cómo diagnosticarías este problema usando las herramientas de observabilidad del módulo. ¿Qué trazas buscarías? ¿Qué indicadores mirarías? ¿Cuál es la causa raíz más probable y cómo la resolverías?

B3. Un equipo propone la siguiente arquitectura para un asistente de recursos humanos: el LLM tiene acceso a una herramienta que consulta directamente la base de datos de RRHH sin filtros de acceso, y el sistema filtra la salida para no mostrar salarios u otros datos sensibles en la respuesta.

Evalúa críticamente esta arquitectura desde el punto de vista de seguridad. ¿Qué riesgo específico introduce? ¿Qué arquitectura alternativa sería más robusta y por qué?

Notas de contraste

Estas notas no son las respuestas "correctas". Son los aspectos más importantes que una respuesta de calidad debería incluir. Úsalas para evaluar la completitud de tus propias

respuestas.

A1: La falla es en el motor RAG o en el proceso de construcción del contexto: el fragmento de precios de 2024 no debería haber pasado al contexto porque no es relevante para la consulta de envíos a Canarias. El problema puede estar en el chunking (el fragmento mezcla información de precios con menciones geográficas) o en el k de la recuperación (se recuperan demasiados fragmentos sin suficiente filtro de relevancia). La solución más robusta incluye re-ranking de fragmentos por relevancia directa y la instrucción al LLM de citar solo los fragmentos que responden directamente la pregunta.

A2: El agente viola el principio de supervisión humana: las acciones con efectos financieros irreversibles (aprobación de un pago) siempre requieren confirmación humana antes de ejecutarse. La instrucción del sistema debe declarar explícitamente: "Nunca ejecutes `aprobar_pago` sin presentar el cálculo completo al operador y recibir confirmación explícita."

A3: El control de acceso diferenciado por rol aplica directamente. La instrucción del sistema debe variar según el rol del usuario: el perfil "enfermero certificado" tiene acceso a información clínica completa; el perfil "estudiante de enfermería" tiene acceso a información clínica de aprendizaje con limitaciones sobre medicamentos controlados. El motor RAG debe filtrar el índice de documentos clínicos por el nivel de autorización del perfil activo.

A4: La ventana de 8.000 tokens se llena con 3.000 de instrucción del sistema más el historial acumulado. Después de ~15 turnos, el historial compite con la instrucción del sistema por el espacio disponible. Cuando el historial excede el espacio restante, el LLM recibe un contexto truncado que puede excluir partes de la instrucción del sistema. Estrategia 1: implementar ventana deslizante con resumen incremental para el historial, limitando a 5 turnos completos + resumen comprimido. Estrategia 2: reducir el tamaño de la instrucción del sistema a lo estrictamente necesario y separar el conocimiento estático en RAG.

A5: Problema 1: violación de privacidad implícita (el usuario no consintió que ese comentario fuera memorizado como dato persistente). Problema 2: el estado emocional momentáneo de una persona no es información útil para sesiones futuras y puede generar sesgos en las respuestas. El diseño correcto del módulo de memoria aplica el criterio de relevancia futura: solo se memoriza información que cambia el comportamiento del sistema de manera que el usuario considera valiosa. Un comentario emocional casual no cumple ese criterio.

B1-B3: Estas preguntas evalúan la capacidad de razonamiento de diseño. No hay respuesta única. Los criterios de calidad son: especificidad (las respuestas se refieren a este caso, no a casos genéricos), consistencia (las decisiones de diseño son coherentes entre sí), y

completitud (se cubren los aspectos de seguridad, observabilidad y operación, no solo la funcionalidad).

Si completaste las cinco preguntas de la Parte A con respuestas que cubren los puntos de las notas de contraste, y al menos dos de las tres preguntas de la Parte B con razonamiento específico y coherente, completaste el módulo con solidez suficiente para abordar el Módulo 4.

Si identificaste preguntas que no pudiste responder bien, eso no es un fracaso: es precisamente la información que necesitas. Las preguntas que no pudiste responder señalan los capítulos que vale la pena releer antes de continuar.

Capítulo 15 — Proyecto Integrador

Sección 15: Próximos pasos y cierre del Módulo 3

Lo que construiste en este módulo

Cuando empezaste el Módulo 3, un sistema de IA era probablemente una caja negra con una API: envías texto, recibes texto. Al cerrar el módulo, esa caja negra tiene partes visibles, nombradas y diseñables.

Ahora sabes que la ventana de contexto es un presupuesto que se administra, no un depósito ilimitado. Que las instrucciones del sistema son el mecanismo principal de control del comportamiento del modelo, y que deben gestionarse como artefactos de software con versiones y responsables. Que la memoria persistente es una selección intencional, no una acumulación automática. Que RAG no es magia: es un pipeline de recuperación con etapas específicas, criterios de calidad y puntos de falla concretos. Que las herramientas le dan agencia al sistema, pero esa agencia requiere controles de acceso y confirmación humana para las acciones irreversibles. Que los agentes son la herramienta correcta cuando la tarea requiere condicionalidad en la secuencia de pasos, no cuando simplemente queremos que el sistema "sea más inteligente". Que observabilidad y seguridad se diseñan desde el primer diagrama de arquitectura, no se agregan como capas después del lanzamiento.

Esos son los instrumentos del Context Engineering profesional. Los tienes. El siguiente módulo los usa.

Qué viene en el Módulo 4

El Módulo 4 se ocupa de las Arquitecturas Modernas de Sistemas de IA. Asume que el Context Engineering del Módulo 3 está dominado y construye sobre esa base.

Los tres saltos de complejidad que el Módulo 4 introduce:

Primero: multiagencia. En el Módulo 3, los agentes son unidades individuales con un conjunto de herramientas. En el Módulo 4, múltiples agentes colaboran en sistemas donde el

trabajo se distribuye, los resultados se sintetizan, y la coordinación entre agentes requiere sus propios mecanismos de comunicación y control. El Context Engineering que diseñaste para un agente único aplica a cada agente del sistema multiagente, pero la arquitectura del sistema completo introduce desafíos nuevos: consistencia de estado entre agentes, gestión de conflictos, y supervisión de sistemas que toman decisiones distribuidas.

Segundo: escala. Los casos del Módulo 3 operan con decenas o cientos de usuarios. Los sistemas de producción a escala empresarial operan con miles o millones. La escala introduce restricciones que cambian las decisiones de diseño: el costo de tokens por millón de interacciones diarias, la latencia bajo carga concurrente alta, la consistencia de la base documental cuando múltiples pipelines de ingestión la modifican simultáneamente, y la operación de sistemas que no pueden tener tiempo de inactividad.

Tercero: infraestructura de IA. El Módulo 3 trató la infraestructura como un requisito (RNF-04: usa servicios de mercado). El Módulo 4 la trata como un dominio de diseño: cómo se seleccionan y combinan los modelos base, cuándo el fine-tuning justifica el costo de entrenamiento, cómo se diseñan los pipelines de evaluación continua que detectan degradaciones del comportamiento del sistema antes de que los usuarios las noten, y cómo se gestiona la evolución de un sistema de IA en producción a lo largo del tiempo.

El Context Engineering del Módulo 3 es el lenguaje que usarás para diseñar y razonar sobre esas arquitecturas más complejas. No lo reemplaza el Módulo 4: lo presupone.

Recursos para continuar

El campo del Context Engineering y la ingeniería de sistemas de IA evoluciona rápidamente. Los conceptos del Módulo 3 son principios que trascienden versiones de modelos y frameworks, pero las implementaciones específicas cambian. Estos recursos te ayudarán a mantenerte actualizado:

Documentación oficial de modelos y APIs. Las páginas de documentación de los proveedores de LLM son la fuente más actualizada sobre capacidades de ventana de contexto, herramientas disponibles, y mejores prácticas de uso de cada modelo específico.

El repositorio de Model Context Protocol (MCP). La especificación oficial de MCP y sus implementaciones de referencia están publicadas en el repositorio de Anthropic. Para cualquier proyecto que use herramientas, la especificación MCP es la referencia definitiva sobre cómo estructurar la comunicación entre orquestador y herramientas.

Evaluaciones y benchmarks públicos de RAG. Los trabajos académicos sobre RAG (RAGAS, BEIR, MTEB) publican métricas de calidad de recuperación que te permiten calibrar si tu pipeline RAG tiene el rendimiento esperable. Son también la mejor fuente para entender las limitaciones de los enfoques actuales.

Comunidades de práctica. Los foros de discusión técnica alrededor de LangChain, LlamaIndex, y los SDKs de los principales proveedores de LLM son donde los equipos de ingeniería comparten problemas reales y soluciones concretas. Los problemas que encontrarás en tus primeros proyectos reales probablemente ya fueron resueltos por alguien en esas comunidades.

Una nota final sobre el oficio

El Context Engineering no es una disciplina de año. Es una disciplina que se construye proyecto a proyecto, y que se mejora con cada sistema que se depura en producción, cada degradación de comportamiento que se diagnostica, y cada decisión de diseño que se revisa a la luz de cómo operó el sistema en la realidad.

Los principios del módulo son una base sólida. Pero la competencia real viene de aplicarlos, equivocarse en los detalles, diagnosticar lo que falló, y volver al diseño con esa experiencia. El checklist de la sección 12 es más útil la tercera vez que lo aplicas que la primera, porque la tercera vez ya sabrás de qué trata cada ítem desde la experiencia, no solo desde la lectura.

El Módulo 4 te espera con esa base construida. Continúa.